


**REV. FR. MOSES ORSHIO ADASU UNIVERSITY, MAKURDI**

**FACULTY OF COMPUTER SCIENCE**

**DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE**

**COURSE OF STUDY: COMPUTER SCIENCE**

# DESIGN AND IMPLEMENTATION OF A WEB-BASED CONSULTANCY SERVICES PORTAL

## (MOAUM CONSULTANCY PORTAL)

**FOR THE CONSULTANCY UNIT, REV. FR. MOSES ORSHIO ADASU UNIVERSITY, MAKURDI**

**A GROUP PROJECT PRESENTATION REPORT**

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS OF THE COURSE

**CMP 472: SOFTWARE ENGINEERING**

**COURSE LECTURER: DR. STEPHEN AKUMA**

(Group members are listed on the next page)

**OCTOBER, 2026**

# GROUP MEMBERS

The following students of the Department of Mathematics and Computer Science jointly designed, implemented, tested and presented the system described in this report.

| S/N | FULL NAME | MATRIC NUMBER | SIGNATURE |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| 8 | | | |
| 9 | | | |
| 10 | | | |
| 11 | | | |
| 12 | | | |
| 13 | | | |
| 14 | | | |
| 15 | | | |

# CERTIFICATION

This is to certify that this group project report, titled **Design and Implementation of a Web-Based Consultancy Services Portal (MOAUM Consultancy Portal) for the Consultancy Unit, Rev. Fr. Moses Orshio Adasu University, Makurdi**, was carried out by the group members listed on the preceding page in partial fulfilment of the requirements of the course CMP 472: Software Engineering, in the Department of Mathematics and Computer Science, Faculty of Computer Science, Rev. Fr. Moses Orshio Adasu University, Makurdi.

…… ……

**Dr. Stephen Akuma** **Date**
Course Lecturer, CMP 472

# ACKNOWLEDGEMENT

The group expresses its sincere gratitude to the course lecturer, **Dr. Stephen Akuma**, whose teaching of software engineering principles in CMP 472 provided the foundation on which this project was planned, designed, built and evaluated.

The group also thanks the Department of Mathematics and Computer Science and the Faculty of Computer Science, Rev. Fr. Moses Orshio Adasu University, Makurdi, for the learning environment that made this work possible, and the Consultancy Unit of the University, whose services (shop allocation, e-learning and summer computer programmes) formed the problem domain of the project.

Finally, each member of the group is acknowledged for the commitment, cooperation and individual contributions that made the successful completion of this project possible.

# ABSTRACT

This report presents the design, implementation, testing and deployment of the **MOAUM Consultancy Portal**, a web application for the Consultancy Unit of Rev. Fr. Moses Orshio Adasu University, Makurdi (formerly Benue State University). The Unit allocates commercial shops on the Western and Eastern Campuses, registers participants for e-learning and summer computer programmes, and offers other consultancy services, processes that are largely paper-based and require repeated physical visits. The aim of the project was to provide a single, secure online portal through which members of the public, students, staff and administrators carry out these processes.

An iterative and incremental development approach was adopted. The system was analysed and modelled using the Unified Modeling Language (use case and class diagrams) and implemented as a full-stack TypeScript application built on the **Next.js 15** framework with **React 19**, styled with **Tailwind CSS** and animated with **Framer Motion**. Data is managed through the **Prisma** object-relational mapper on **SQLite** during development and **PostgreSQL (Neon)** in production; authentication and role-based access control are provided by **NextAuth.js**. The portal integrates **Paystack** for online payment, **Gmail SMTP** (via Nodemailer) for email notifications, **Vercel Blob** for document storage and **pdf-lib** for generating official allocation and admission letters. The application was deployed on **Vercel** with continuous deployment from a **GitHub** repository.

The completed system supports online shop applications with document upload and public status tracking; programme registration with bank-transfer or card payment; dashboards for applicants; and an administrative console for reviewing applications, allocating shops, verifying payments, managing users and exporting reports.

Functional, security and responsiveness tests were carried out on all major features; every executed test passed after the defects found during testing were corrected. The portal is live at https://moaum-consultancy.vercel.app.

**Keywords:** web portal, shop allocation, programme registration, Next.js, role-based access control, online payment, software engineering.

# TABLE OF CONTENTS

GROUP MEMBERS ..... ii

CERTIFICATION ..... iii

ACKNOWLEDGEMENT ..... iv

ABSTRACT ..... v

LIST OF FIGURES ..... xi

LIST OF TABLES ..... xiii

LIST OF ABBREVIATIONS ..... xiv

CHAPTER ONE INTRODUCTION ..... 1

1.1 Background of the Study ..... 1

1.2 Statement of the Problem ..... 2

1.3 Aim and Objectives ..... 2

1.4 Scope of the Project ..... 3

1.5 Significance of the Project ..... 3

1.6 Limitations of the Project ..... 4

1.7 Definition of Terms ..... 4

CHAPTER TWO SYSTEM ANALYSIS AND DESIGN ..... 6

2.1 Introduction ..... 6

2.2 Analysis of the Existing Process ..... 6

2.3 Requirements Specification ..... 6

2.3.1 Functional Requirements ..... 6

2.3.2 Non-Functional Requirements ..... 7
2.4 Development Methodology ..... 8
2.5 System Architecture ..... 8
2.6 Use Case Model ..... 9
2.7 Class Model ..... 10
2.8 Database Design ..... 11
2.9 Security Design ..... 12
CHAPTER THREE TOOLS AND TECHNOLOGIES ..... 14
3.1 Introduction ..... 14
3.2 Programming Languages ..... 14
3.3 Frameworks and Libraries ..... 14
3.4 Database and Storage ..... 15
3.5 External Services ..... 16
3.6 Development, Testing and Documentation Tools ..... 16
CHAPTER FOUR IMPLEMENTATION, TESTING AND DEPLOYMENT ..... 17
4.1 Introduction ..... 17
4.2 Implementation of the Modules ..... 17
4.2.1 Public Website and Visual Identity ..... 17
4.2.2 User Accounts and Authentication ..... 19
4.2.3 Shop Allocation Module ..... 19
4.2.4 Programme Registration and Payment Module ..... 22

4.2.5 Applicant Dashboard ..... 24

4.2.6 Administrative Console ..... 25

4.2.7 Email Notifications ..... 28

4.2.8 Official Letters (PDF) ..... 28

4.3 Testing ..... 29

4.4 Deployment ..... 31

4.5 Discussion of Results ..... 32

CHAPTER FIVE SUMMARY, CONCLUSION AND RECOMMENDATIONS ..... 33

5.1 Summary ..... 33

5.2 Conclusion ..... 33

5.3 Recommendations ..... 33

5.4 Suggestions for Further Work ..... 34

REFERENCES ..... 35

APPENDIX A: SELECTED SOURCE CODE ..... 36

A.1 Database Schema ..... 36

A.2 Input Validation ..... 38

A.3 Authentication and Role Checks ..... 39

A.4 Route Protection ..... 40

A.5 Submitting a Shop Application ..... 40

A.6 Reviewing and Allocating ..... 41

A.7 Verifying Online Payments ..... 42

A.8 Document Storage ..... 43

A.9 Letter Generation ..... 43

A.10 Sending Email ..... 44

A.11 Deployment Build Step ..... 44

APPENDIX B: ADDITIONAL SCREENSHOTS ..... 47

APPENDIX C: SAMPLE GENERATED LETTER ..... 50

APPENDIX D: TEST CASES AND RESULTS ..... 51

APPENDIX E: PROJECT STRUCTURE ..... 53

APPENDIX F: INSTALLATION AND DEPLOYMENT GUIDE ..... 54

F.1 Running the Portal Locally ..... 54

F.2 Demonstration Accounts (local only) ..... 54

F.3 Configuration Settings ..... 54

F.4 Deploying to Vercel ..... 55

# LIST OF FIGURES

Figure 2.1: System architecture of the MOAUM Consultancy Portal ..... 9

Figure 2.2: Use case diagram of the MOAUM Consultancy Portal ..... 10

Figure 2.3: Class diagram of the MOAUM Consultancy Portal ..... 11

Figure 4.1: Home page of the portal ..... 18

Figure 4.2: Home page on a mobile phone ..... 18

Figure 4.3: Sign-in page ..... 19

Figure 4.4: Shop availability on the Western and Eastern Campuses ..... 20

Figure 4.5: Online shop application form with document upload ..... 21

Figure 4.6: Public status tracking by reference number ..... 22

Figure 4.7: Programme details and registration ..... 23

Figure 4.8: Registrations with online payment and bank-transfer options ..... 24

Figure 4.9: Applicant dashboard ..... 25

Figure 4.10: Applicant's shop applications with status timeline ..... 25

Figure 4.11: Admin console overview and statistics ..... 26

Figure 4.12: Shop applications awaiting review ..... 27

Figure 4.13: Reviewing an application and allocating a shop ..... 27

Figure 4.14: Verifying payments and confirming registrations ..... 28

Figure 4.15: Generated letter of shop allocation (PDF) ..... 29

Figure B.1: Home page – services overview ..... 47

Figure B.2: Programmes catalogue ..... 47

Figure B.3: Contact page and enquiry form ..... 48

Figure B.4: Admin console – shops register ..... 48

Figure B.5: Admin console – user management (administrators only) ..... 49

Figure C.1: Generated letter of admission (PDF) ..... 50

# LIST OF TABLES

Table 1.1: Definition of terms ..... 4

Table 2.1: Functional requirements ..... 6

Table 2.2: Non-functional requirements ..... 7

Table 2.3: Database tables ..... 11

Table 3.1: Programming and markup languages ..... 14

Table 3.2: Frameworks and libraries used ..... 14

Table 3.3: Database and storage technologies ..... 15

Table 3.4: External services ..... 16

Table 3.5: Development and testing tools ..... 16

Table 4.1: Summary of test results ..... 30

Table 4.2: Defects found during testing and their correction ..... 31

Table D.1: Test cases and results ..... 51

Table F.1: Demonstration accounts ..... 54

Table F.2: Environment variables ..... 54

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| CSS | Cascading Style Sheets |
| CSV | Comma-Separated Values |
| HTML | HyperText Markup Language |
| HTTPS | HyperText Transfer Protocol Secure |
| JWT | JSON Web Token |
| MOAUM | Rev. Fr. Moses Orshio Adasu University, Makurdi |
| ORM | Object-Relational Mapper / Mapping |
| PDF | Portable Document Format |
| RBAC | Role-Based Access Control |
| SMTP | Simple Mail Transfer Protocol |
| SQL | Structured Query Language |
| UI / UX | User Interface / User Experience |
| UML | Unified Modeling Language |
| URL | Uniform Resource Locator |

# CHAPTER ONE
# INTRODUCTION

## 1.1 Background of the Study

Rev. Fr. Moses Orshio Adasu University, Makurdi (MOAUM), formerly Benue State University, is a state-owned university in Makurdi, Benue State, Nigeria; the institution was renamed in 2025 (Wikipedia contributors, 2026). Like many Nigerian universities, it operates a **Consultancy Unit**, the commercial and outreach arm of the University, which generates internally generated revenue by putting the University's facilities and expertise at the service of students, staff and the wider public.

The Unit's core services, as captured in the project brief, are: the **allocation of commercial shops** (business centres, restaurants, retail units, service outlets and kiosks) on the Western and Eastern Campuses; **programme registration** for e-learning courses and Summer Computer Programmes such as Website Design and Microsoft Office Proficiency; and **other consultancy services** such as career guidance, training workshops, facility booking and research advisory.

These services involve many applicants, supporting documents, approvals and payments. Software engineering offers a disciplined way of turning such a business process into a reliable information system: requirements are elicited and specified, the system is modelled, implemented in increments, verified through testing and finally deployed and maintained (Sommerville, 2016; Pressman & Maxim, 2020). This project applies that discipline, as taught in CMP 472: Software Engineering, to build a complete, working web portal for the Consultancy Unit.

## 1.2 Statement of the Problem

Prior to this project, the Unit's processes were assumed to rely on paper forms, notice boards and in-person visits. This creates the following problems, which the system was designed to solve:

1. **No single source of information.** Prospective applicants cannot easily see which shops are available on each campus, what programmes are running, or what each costs.
2. **Slow, opaque allocation.** Paper applications are difficult to track; applicants must return repeatedly to learn the status of their application, and decisions are hard to audit.
3. **Manual registration and payment reconciliation.** Matching bank transfers to programme registrations by hand is slow and error-prone.
4. **Document handling.** Physical copies of identification documents and business proposals can be misplaced and are not easily shared with the reviewing committee.
5. **No management information.** The Unit has no immediate view of occupancy, pending applications, registrations or revenue.

## 1.3 Aim and Objectives

The **aim** of the project is to design and implement a secure, responsive web portal through which the Consultancy Unit manages shop allocation, programme registration and related consultancy services online.

The specific **objectives** are to:

1. analyse the Unit's processes and model the required system using UML use case and class diagrams;

2. implement a public website presenting the Unit's services, live shop availability and programme catalogue;
3. implement online shop applications for the Western and Eastern Campuses with document upload and public status tracking by reference number;
4. implement programme registration with payment by bank transfer and by card through Paystack;
5. provide dashboards for applicants and an administrative console with role-based access for staff and administrators;
6. automatically generate official allocation and admission letters in PDF and send email notifications at each stage; and
7. test the system and deploy it to a publicly accessible production environment.

### 1.4 Scope of the Project

The project covers the full web portal: the public website (home, services, shop allocation, programmes, about and contact pages); user registration and authentication; shop applications and allocation for both campuses; e-learning and summer programme registration and payment; applicant dashboards; the administrative console (application review and allocation, shops register, registration and payment verification, enquiries, users and CSV reports); email notifications; PDF letters; and deployment. Payroll, accounting and the University's main student information system are outside the scope of this project.

### 1.5 Significance of the Project

For **applicants**, the portal removes the need for repeated visits: they can browse, apply, upload documents, pay and track progress from any device. For the **Consultancy Unit**, it provides a transparent, auditable workflow, automatic notifications and letters,

and real-time statistics on occupancy, applications and revenue. For the **University**, it supports internally generated revenue and presents the Unit professionally online. Academically, the project demonstrates the application of software engineering principles (requirements analysis, modelling, iterative implementation, testing and deployment) to a real institutional problem.

### 1.6 Limitations of the Project

- Contact details, staff names, testimonials and the bank account shown on the site are placeholders pending confirmation by the Consultancy Unit.
- Online card payment runs in Paystack **test mode**; live payments require the Unit's Paystack business account to be verified.
- Email is sent through a Gmail account, which limits sending to about 500 messages per day; a dedicated email service is recommended for high volume.
- Uploaded documents are limited to 2 MB per file because of the hosting platform's request-size limit.

### 1.7 Definition of Terms

**Table 1.1: Definition of terms**

| Term | Definition |
|---|---|
| Consultancy Unit | The commercial and outreach arm of the University that provides the services managed by the portal. |
| Shop allocation | The process of reviewing applications and assigning a campus commercial space to an applicant. |
| Reference number | A unique code such as MOAUM-SA-2026-4F7K2 (shop application) or MOAUM-PR-2026-8H2LQ (programme registration) used to track a record. |
| Role | The level of access of a user account: Student (applicant), Staff or Administrator. |
| Server Action | A Next.js function that runs on the server and is called directly from a form or button in the browser. |
| ORM | A library (Prisma) that maps database tables to program objects so that queries are written in TypeScript instead of raw SQL. |

| Term | Definition |
| --- | --- |
| Webhook | An HTTP request sent by an external service (Paystack) to notify the portal that an event, such as a successful payment, has occurred. |

# CHAPTER TWO
# SYSTEM ANALYSIS AND DESIGN

## 2.1 Introduction

This chapter analyses the existing process, specifies the requirements of the new system and describes its design: the development methodology, the overall architecture, the UML models, the database design and the security design.

## 2.2 Analysis of the Existing Process

In the existing process, an applicant for a campus shop obtains a form from the Unit's office, completes it, attaches photocopies of an identification document and a business proposal and submits them in person. The allocation committee reviews the files, and the applicant must return to learn the outcome before collecting an allocation letter and paying rent at the Bursary. Programme registration follows a similar pattern, with bank-teller evidence presented physically for verification. The weaknesses identified are summarised in Section 1.2: poor visibility of availability, slow and opaque decisions, manual payment reconciliation, vulnerable paper documents and an absence of management information.

## 2.3 Requirements Specification

### *2.3.1 Functional Requirements*

**Table 2.1: Functional requirements**

| ID | Requirement | Actor |
|---|---|---|
| FR1 | Display the Unit's services, campus shop availability and the programme catalogue to the public. | Visitor |
| FR2 | Allow a visitor to create an account and sign in with email and password. | Visitor |
| FR3 | Allow an applicant to apply for a shop on either campus, choose a preferred shop and upload an ID and a business proposal. | Student |
| FR4 | Issue a unique reference number and allow anyone to track an application or registration by that reference. | Visitor, Student |

| ID | Requirement | Actor |
|---|---|---|
| FR5 | Allow registration for e-learning and summer programmes, enforcing the number of seats. | Student |
| FR6 | Accept payment by bank transfer (with a transaction reference) or online through Paystack. | Student |
| FR7 | Provide an applicant dashboard showing applications, registrations, status timelines and letters. | Student |
| FR8 | Allow staff to review applications, record decisions with notes and allocate shops. | Staff |
| FR9 | Allow staff to maintain the shops register and change shop status. | Staff |
| FR10 | Allow staff to verify payments and confirm, reject or complete registrations. | Staff |
| FR11 | Generate allocation and admission letters as PDF documents. | System |
| FR12 | Send email notifications when applications and registrations are received or updated, and for contact enquiries. | System |
| FR13 | Show statistics and export applications and registrations as CSV. | Staff |
| FR14 | Allow administrators to change user roles and activate or deactivate accounts. | Administrator |

### *2.3.2 Non-Functional Requirements*

**Table 2.2: Non-functional requirements**

| Attribute | Requirement |
|---|---|
| Security | Passwords hashed; every protected page, action and file checked for the user's role and ownership; payment results verified with the payment provider. |
| Usability | Clear navigation, inline form validation, toast feedback and a consistent visual identity based on the University's colours and crest. |
| Responsiveness | Usable on phones (375 px wide) through to desktop screens without horizontal scrolling. |
| Performance | Pages rendered on the server and visible before JavaScript finishes loading, for slow connections. |
| Reliability | A failed email must never undo or block the action that triggered it; duplicate payment notifications must be harmless. |
| Maintainability | Type-checked code (TypeScript), a single schema for the database, and documented setup and deployment. |
| Portability | SQLite for local development and PostgreSQL in production without code changes. |

## 2.4 Development Methodology

An **iterative and incremental** approach, in the spirit of agile development (Beck et al., 2001), was followed. Rather than building the whole system before testing, the portal was delivered in a sequence of working increments, each of which was implemented, type-checked, built and tested in a browser before the next began:

1. **Foundation:** project scaffolding, database schema, seed data, authentication and the shared user-interface components.
2. **Public website:** layout, home page, services, about and contact pages.
3. **Core business modules:** shop allocation and programme registration, with document upload and status tracking.
4. **Dashboards:** the applicant dashboard and the administrative console.
5. **Integrations:** email notifications, PDF letters and Paystack online payment.
6. **Deployment:** preparation for cloud hosting, then continuous deployment to Vercel.
7. **Refinement:** adoption of the official University crest, and documentation (UML diagrams and this report).

Source code was version-controlled with Git from the first increment and published to GitHub; each completed increment was recorded as a commit, and after deployment every commit to the main branch was automatically built and released. Development was carried out with the assistance of an AI coding tool, Claude Code (Anthropic, 2026), which the group directed and whose output was reviewed, tested and corrected at each increment (see Section 4.3).

## 2.5 System Architecture

The portal follows a **layered architecture** (Figure 2.1). The **presentation layer** consists of three areas in the browser: the public website, the applicant dashboard and

the administrative console. The **application layer** is a Next.js 15 application in which pages are rendered on the server as React Server Components; data changes are performed by **Server Actions**, while file downloads, PDF letters, CSV reports and payment callbacks are served by **API routes**. A middleware component enforces authentication before any dashboard or console page is reached. The **data layer** is a relational database accessed only through the Prisma ORM, together with object storage for uploaded documents. **External services** provide payment (Paystack) and email delivery (Gmail SMTP).

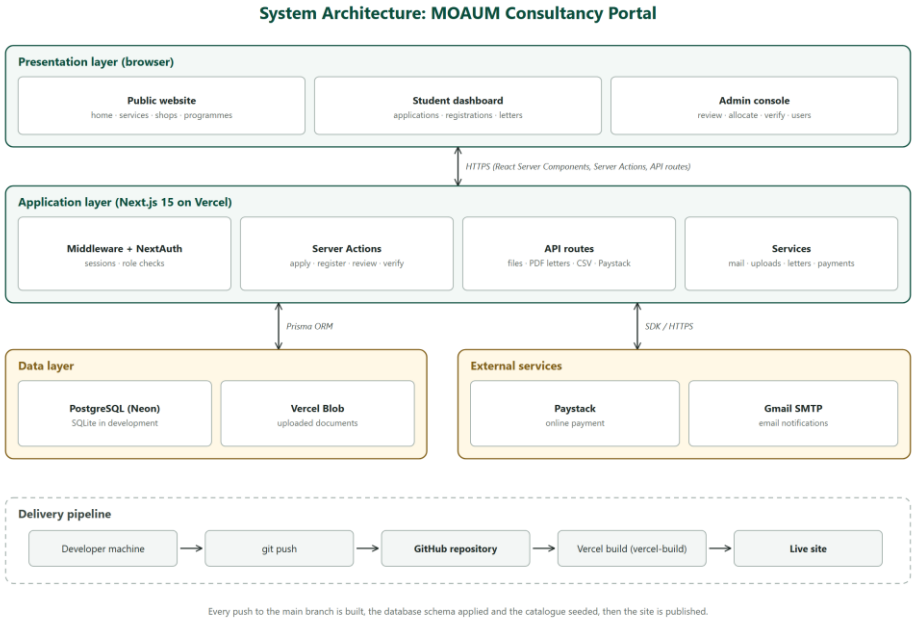



*Figure 2.1: System architecture of the MOAUM Consultancy Portal*

## 2.6 Use Case Model

Figure 2.2 presents the use case diagram, drawn according to the UML 2.5.1 notation (Object Management Group, 2017; Fowler, 2003). Four human actors interact with the system. The **Visitor** (any member of the public) can browse services and programmes, view shop availability, track an application, submit an enquiry, create an account and log in. The **Student / Applicant** is a specialised Visitor who can also apply for a shop, register for a programme, pay a programme fee, view the dashboard,

download letters and manage a profile. The **Staff** member reviews and allocates applications, manages the shops register, verifies payments, reads enquiries and views statistics. The **Administrator** is a specialised Staff member who additionally manages users and roles. Two external systems participate: the Email Service sends notifications and Paystack processes online payments.

«include» relationships show behaviour that always occurs as part of another use case (for example, *Apply for Shop* always includes *Upload Documents* and *Send Email Notification*), while «extend» relationships show optional behaviour (for example, *Pay Online (Paystack)* and *Submit Transfer Reference* are alternative ways of completing *Pay Programme Fee*).

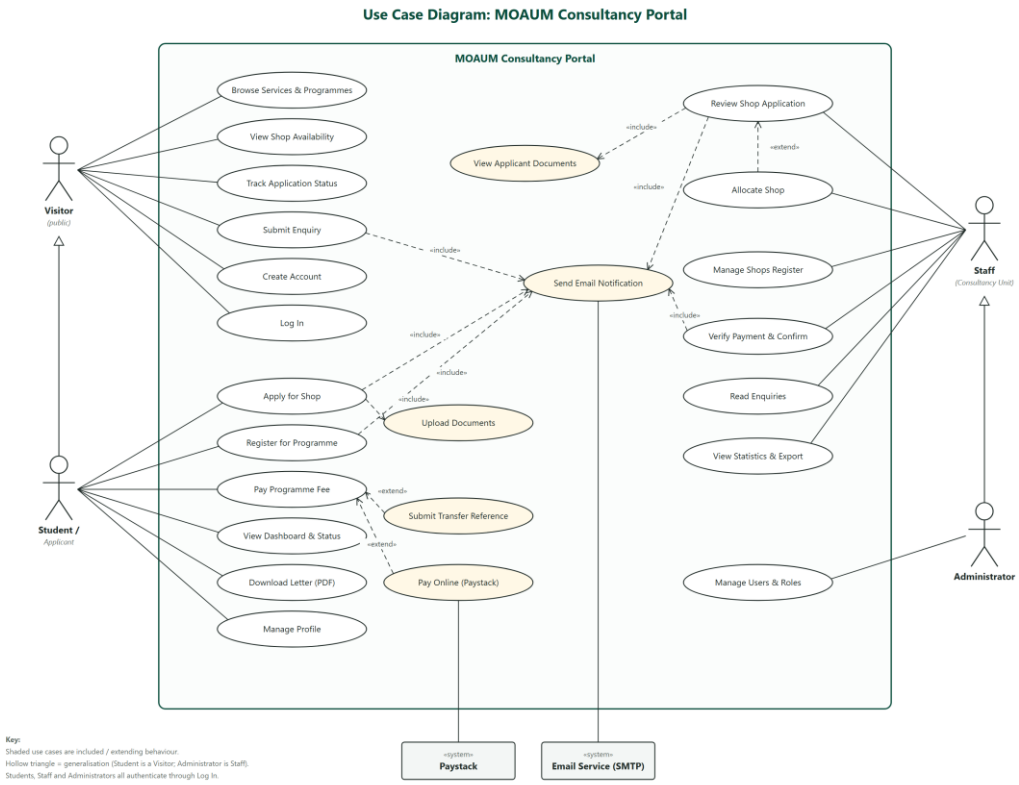



*Figure 2.2: Use case diagram of the MOAUM Consultancy Portal*

## 2.7 Class Model

Figure 2.3 shows the class diagram derived directly from the implemented data model. Six persistent classes are defined. A **User** applies for zero or more

**ShopApplications**, and a staff user reviews them; each ShopApplication may name one preferred **Shop** and may be allocated one Shop. A User makes zero or more **Registrations**, each for exactly one **Programme**. **ContactMessage** stores enquiries from the contact form. Status and category attributes take their values from the enumerations shown at the bottom of the diagram, and operations marked «staff» or «admin» may only be performed by users with that role.

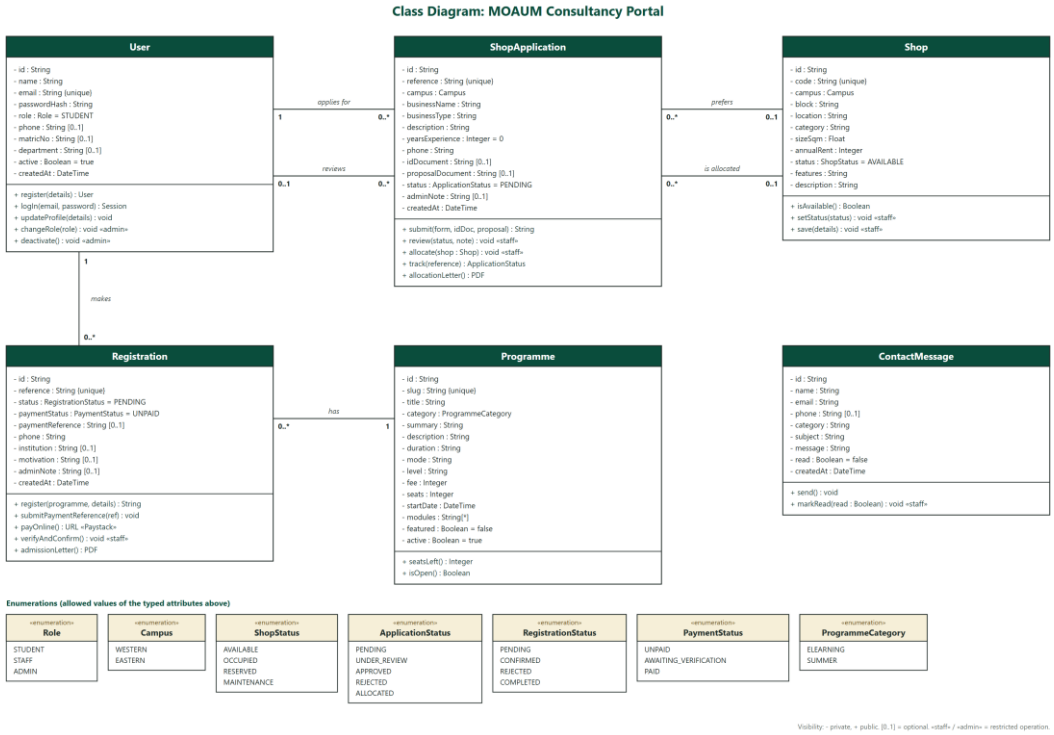



*Figure 2.3: Class diagram of the MOAUM Consultancy Portal*

## 2.8 Database Design

The database schema is defined once in the Prisma schema language (Appendix A, Listing A.1) and is used to create the tables in both SQLite and PostgreSQL. Table 2.3 summarises the tables and their relationships.

**Table 2.3: Database tables**

| Table | Purpose | Key fields and relationships |
|---|---|---|
| User | Accounts for applicants, staff and administrators | email (unique), passwordHash, role, active; one-to-many with ShopApplication and Registration |

| Table | Purpose | Key fields and relationships |
|---|---|---|
| Shop | The register of commercial spaces on both campuses | code (unique, e.g. WC-A01), campus, category, sizeSqm, annualRent, status |
| ShopApplication | An application for a shop | reference (unique), status, document paths; foreign keys applicantId, preferredShopId, allocatedShopId, reviewedById |
| Programme | An e-learning or summer programme | slug (unique), category, fee, seats, startDate, modules |
| Registration | A participant's registration for a programme | reference (unique), status, paymentStatus, paymentReference; userId and programmeId (unique together) |
| ContactMessage | An enquiry from the contact form | category, subject, message, read |

Two design decisions are worth noting. First, every application and registration receives a short, human-readable **reference number** (for example MOAUM-SA-2026-4F7K2) generated from an alphabet that excludes easily confused characters, so applicants can quote it at the office or type it into the tracking tool. Second, the pair (userId, programmeId) on Registration is declared **unique**, so the database itself prevents a participant from registering twice for the same programme.

### 2.9 Security Design

- **Authentication:** passwords are hashed with bcrypt before storage; sessions are signed JSON Web Tokens issued by NextAuth.js and valid for seven days.
- **Role-based access control:** a middleware requires a session for every /dashboard and /admin page; the console layout admits only Staff and Administrators; user management is restricted to Administrators; and every Server Action and API route re-checks the user's role and ownership, so protection does not depend on the user interface alone.
- **Immediate revocation:** on every request the session is re-validated against the database, so a deactivated or deleted account loses access immediately rather than when its token expires.

- **Document privacy:** uploaded files are stored outside the public web folder (or in private-by-obscurity cloud storage) and are only streamed through a route that checks that the viewer owns the application or is staff.
- **Input validation:** every form is validated with the same Zod schema in the browser and again on the server; uploads are restricted to PDF, JPG and PNG files of at most 2 MB.
- **Payment integrity:** payment results are never trusted from the browser. The server re-verifies each transaction with Paystack, checks the amount and currency against the fee on record and accepts webhooks only with a valid HMAC-SHA512 signature.
- **Protection against common web risks** identified by the OWASP Top 10 (OWASP Foundation, 2021), including broken access control, injection (all queries are parameterised by Prisma) and server-side request forgery (stored file URLs are only fetched from the storage provider's domain).

# CHAPTER THREE
# TOOLS AND TECHNOLOGIES

## 3.1 Introduction

This chapter gives a breakdown of every language, framework, library, service and tool used to build, test and deploy the portal, with the exact versions used and the role each plays in the system. All of the tools listed are free or have a free tier sufficient for the project.

## 3.2 Programming Languages

**Table 3.1: Programming and markup languages**

| Language | Where it was used |
|---|---|
| TypeScript 5.9 | All application code: pages, components, Server Actions, API routes, database access and scripts. Static typing caught errors before the code ran. |
| JavaScript (ES2022+) | Build and deployment scripts (for example prepare-postgres.mjs) and the configuration files. |
| HTML and CSS | Generated from React components (JSX) and Tailwind CSS utility classes; also used for the HTML email templates. |
| Prisma Schema Language | Definition of the database tables, fields and relationships. |
| SQL | Generated by Prisma for SQLite and PostgreSQL; not written by hand. |
| Python 3 | Generation of the UML and architecture diagrams used in this report. |

## 3.3 Frameworks and Libraries

Table 3.2 lists the frameworks and libraries that make up the application, grouped by purpose.

**Table 3.2: Frameworks and libraries used**

| Tool (version) | Category | Purpose in the portal |
|---|---|---|
| Next.js (15.5) | Full-stack framework | Routing, server-side rendering, Server Actions, API routes, image optimisation and the production build (Vercel, 2026a). |
| React (19.3) | UI library | Building the user interface from reusable components (Meta Platforms, 2026). |

| Tool (version) | Category | Purpose in the portal |
|---|---|---|
| Tailwind CSS (3.4) | Styling | Utility-first styling, responsive layout and the light/dark theme (Tailwind Labs, 2024). |
| shadcn/ui pattern + Radix UI (1.x–2.x) | UI components | Accessible dialogs, tabs, dropdown menus and accordions styled to the University's colours. |
| Framer Motion (12.43) | Animation | Page transitions, scroll-reveal effects, animated counters, the parallax hero and micro-interactions. |
| Lucide React (0.468) | Icons | Consistent line icons throughout the interface. |
| React Hook Form (7.90) + Zod (3.25) | Forms and validation | Form state and validation rules shared by the browser and the server. |
| Prisma ORM (5.22) | Database access | Type-safe queries, schema management (db push) and seeding (Prisma Data, 2024). |
| NextAuth.js (4.24) | Authentication | Email/password sign-in, JWT sessions and role information in the session (NextAuth.js, 2024). |
| bcryptjs (2.4) | Security | One-way hashing of passwords. |
| Nodemailer (7.0) | Email | Sending HTML notification emails over SMTP. |
| pdf-lib (1.17) | Documents | Generating the A4 allocation and admission letters with the University crest. |
| @vercel/blob (2.8) | File storage | Storing uploaded ID documents and proposals in production. |
| Sonner (1.7) and next-themes (0.4) | User feedback and theming | Animated toast notifications; light/dark mode switching. |
| class-variance-authority, clsx, tailwind-merge | Styling utilities | Component variants and safe merging of CSS classes. |

## 3.4 Database and Storage

**Table 3.3: Database and storage technologies**

| Technology | Environment | Role |
|---|---|---|
| SQLite | Development | A single-file database that needs no server, used on the developers' computers. |
| PostgreSQL on Neon (serverless) | Production | The live relational database, connected through the Vercel Storage integration. |
| Local uploads folder | Development | Storage for uploaded documents outside the public web folder. |
| Vercel Blob | Production | Cloud object storage for uploaded documents. |

## 3.5 External Services

**Table 3.4: External services**

| Service | Purpose |
|---|---|
| Paystack | Online payment of programme fees by card, bank transfer or USSD; the integration uses Paystack's transaction initialise and verify endpoints and signed webhooks (Paystack, 2026). |
| Gmail SMTP | Delivery of notification emails, authenticated with a Google App Password. |
| Vercel | Hosting of the production application with automatic builds from GitHub (Vercel, 2026b). |
| GitHub | Remote Git repository and the trigger for continuous deployment. |

## 3.6 Development, Testing and Documentation Tools

**Table 3.5: Development and testing tools**

| Tool | Purpose |
|---|---|
| Node.js (24) and npm (11) | JavaScript runtime and package manager used to install dependencies and run the application. |
| Git | Version control of all source code. |
| Claude Code (Anthropic) | AI-assisted development environment used under the group's direction to write, refactor and test code, operate the browser for testing and prepare documentation. |
| TypeScript compiler (tsc) | Static type checking before every build. |
| tsx | Running TypeScript scripts directly (database seeding, email test, payment tests). |
| Microsoft Edge (headless) and a built-in test browser | Functional testing of pages and flows, layout checks at mobile and desktop widths, and capture of the screenshots in this report. |
| Prisma CLI | Applying the schema to the database and loading seed data. |
| Python with PyMuPDF | Rendering generated PDF letters to images for visual verification. |
| Microsoft Word | Final formatting of this report. |

# CHAPTER FOUR
# IMPLEMENTATION, TESTING AND DEPLOYMENT

## 4.1 Introduction

This chapter describes how each module of the portal was implemented, presents screenshots of the working system, reports the testing carried out and the defects corrected, and explains how the system was deployed. The finished code base comprises **101 source files** and about **7,700 lines** of TypeScript, styles and schema, organised as shown in Appendix E.

## 4.2 Implementation of the Modules

### *4.2.1 Public Website and Visual Identity*

The public website consists of the home, services, shop allocation, programmes, about and contact pages. The visual identity uses the University's deep green (#0A4D3C) with gold accents (#C9A227), the official University crest, the Inter and Fraunces typefaces, and both light and dark themes. The home page (Figure 4.1) combines a parallax hero section with service cards, animated statistics, live campus shop availability drawn from the database, featured programmes, a four-step process guide and testimonials. All pages are rendered on the server, and entrance animations use CSS so that content remains visible on slow connections before JavaScript loads. The layout adapts to phones (Figure 4.2).

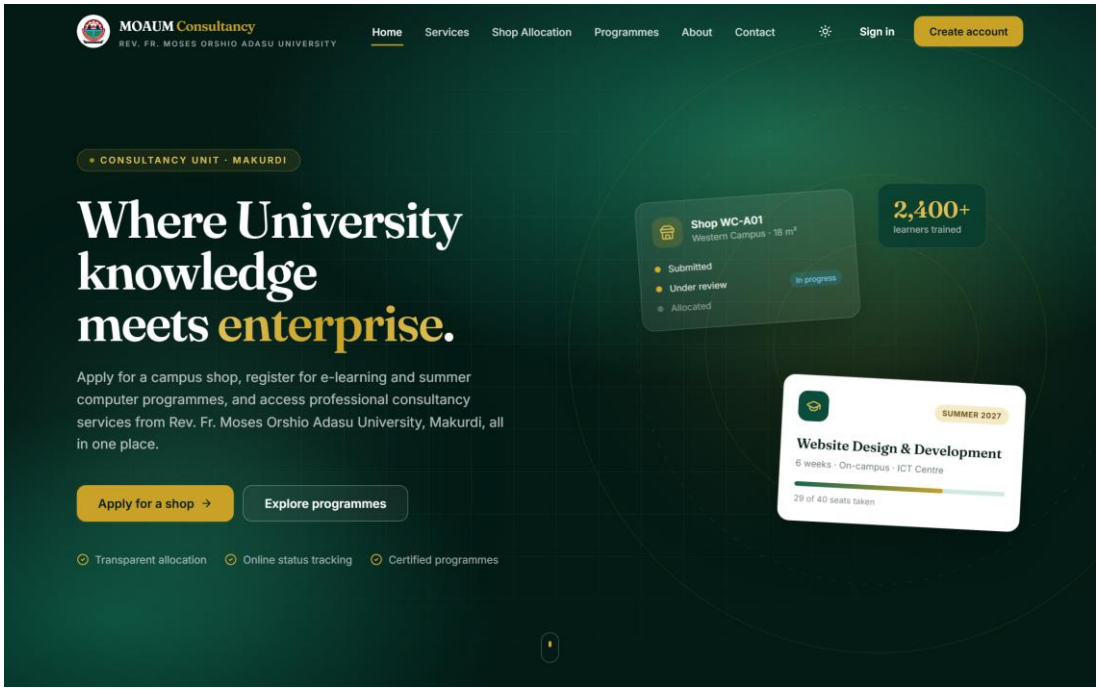



*Figure 4.1: Home page of the portal*

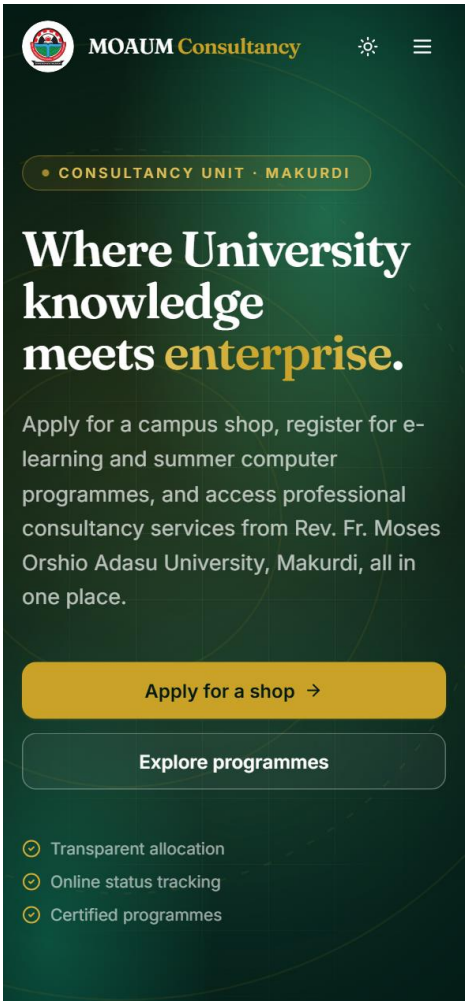



*Figure 4.2: Home page on a mobile phone*

### *4.2.2 User Accounts and Authentication*

Visitors create an account with their name, email, phone number and, optionally, matriculation or staff number and department. Passwords must be at least eight characters and contain a letter and a digit. Sign-in (Figure 4.3) is handled by NextAuth.js; after signing in, users are returned to the page they were trying to reach. New accounts are given the Student role; Staff and Administrator roles are assigned by an administrator.

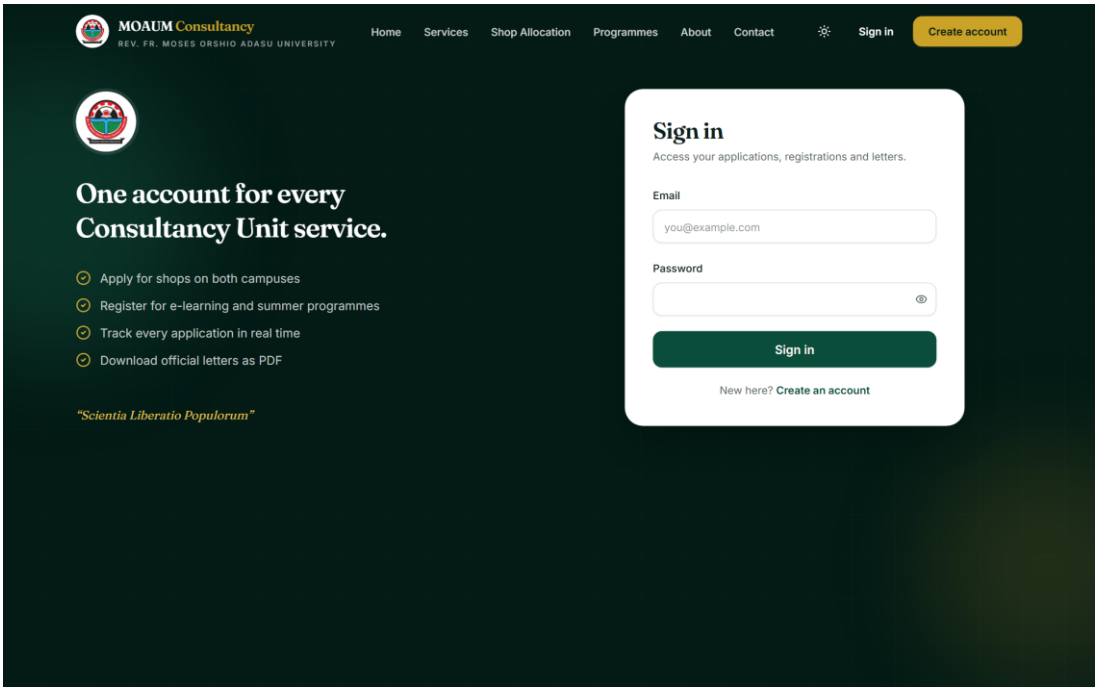



*Figure 4.3: Sign-in page*

### *4.2.3 Shop Allocation Module*

The shop allocation page lists every shop on the register with tabs for the **Western** and **Eastern Campuses**, category filters and an "available only" switch (Figure 4.4). Each card shows the shop code, location, size, annual rent, facilities and live status. Selecting **Apply for this shop** scrolls to the application form and pre-selects that shop.

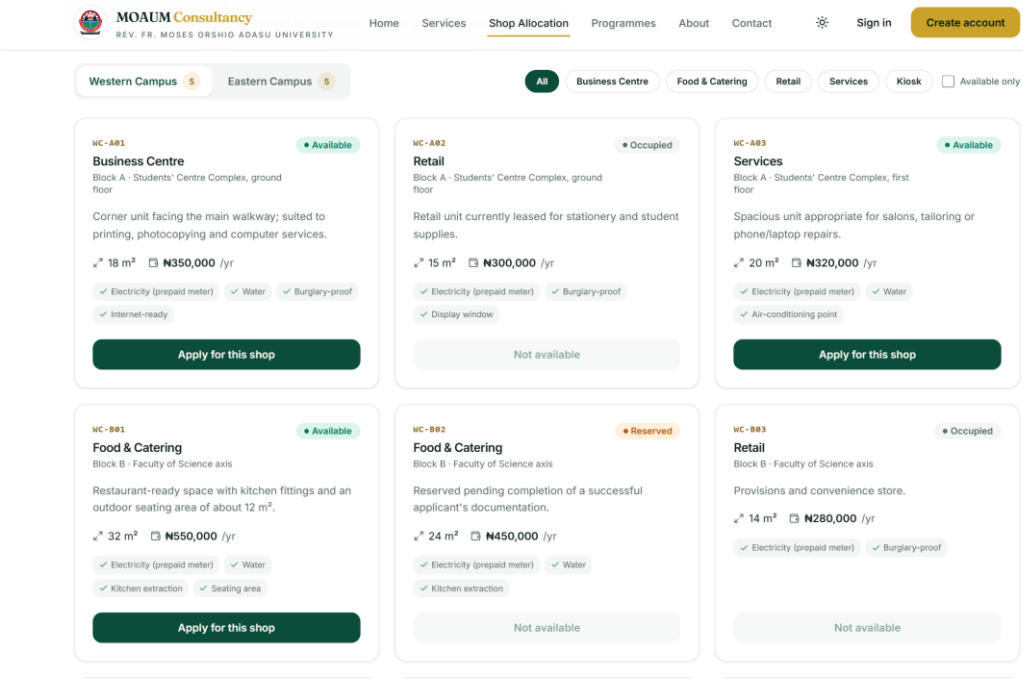



*Figure 4.4: Shop availability on the Western and Eastern Campuses*

The application form (Figure 4.5) collects the business details and two documents, a means of identification and a business proposal, as PDF, JPG or PNG files of up to 2 MB each. On submission the Server Action `submitShopApplication` (Appendix A, Listing A.5) validates the data again on the server, rejects a second active application for the same campus, confirms that the chosen shop is still available, stores the documents, creates the record with a unique reference and emails the applicant.

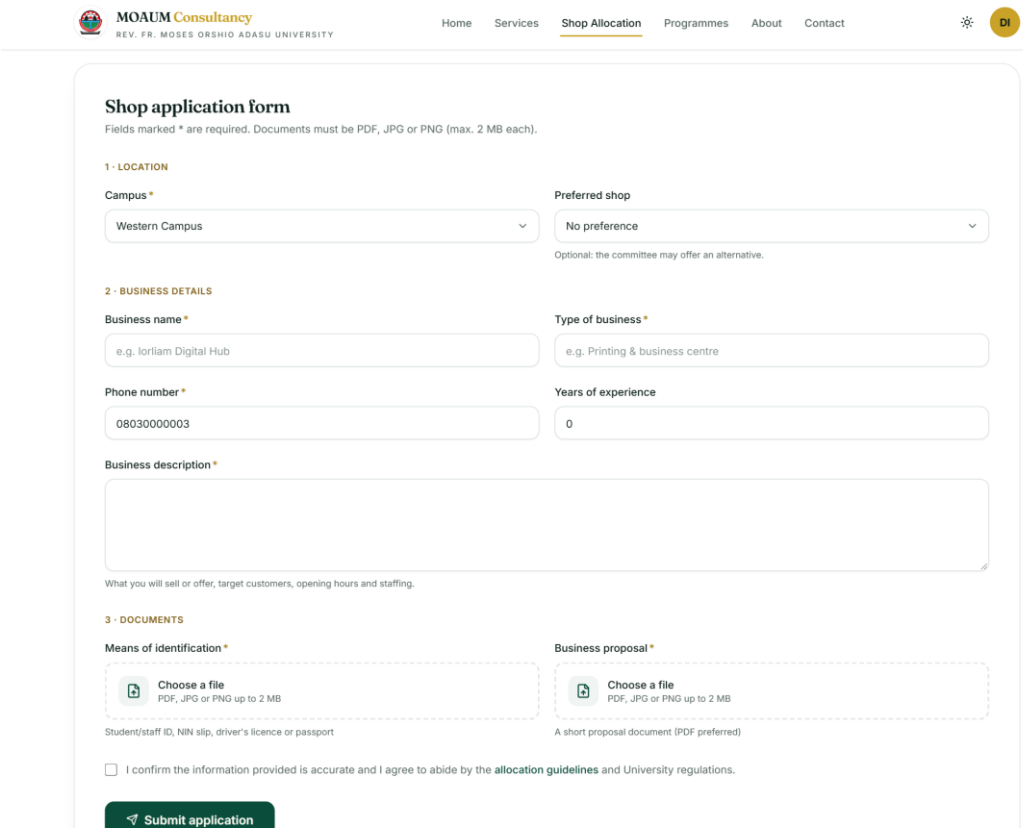



*Figure 4.5: Online shop application form with document upload*

Anyone holding a reference number can check its progress with the tracking tool on the same page (Figure 4.6), which shows a status timeline (Submitted, Under review, Approved, Allocated, or Rejected) and any note from the committee.

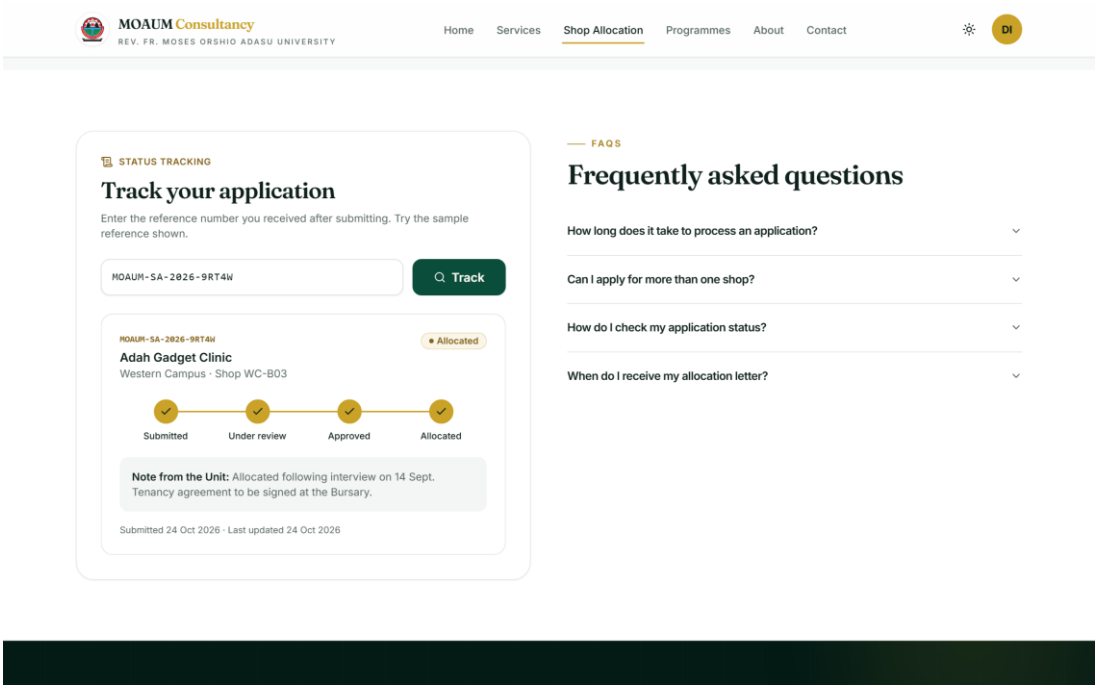



*Figure 4.6: Public status tracking by reference number*

### *4.2.4 Programme Registration and Payment Module*

The programmes page lists the e-learning courses and Summer Computer Programmes (Website Design & Development, Microsoft Office Proficiency, Graphic Design Essentials, Data Analysis with Excel & Python, and Computer Hardware & Networking), each with duration, mode, start date, fee and seats remaining. A programme's page (Figure 4.7) shows its description and modules beside the registration form; registration is refused when the programme is full or the user is already registered.

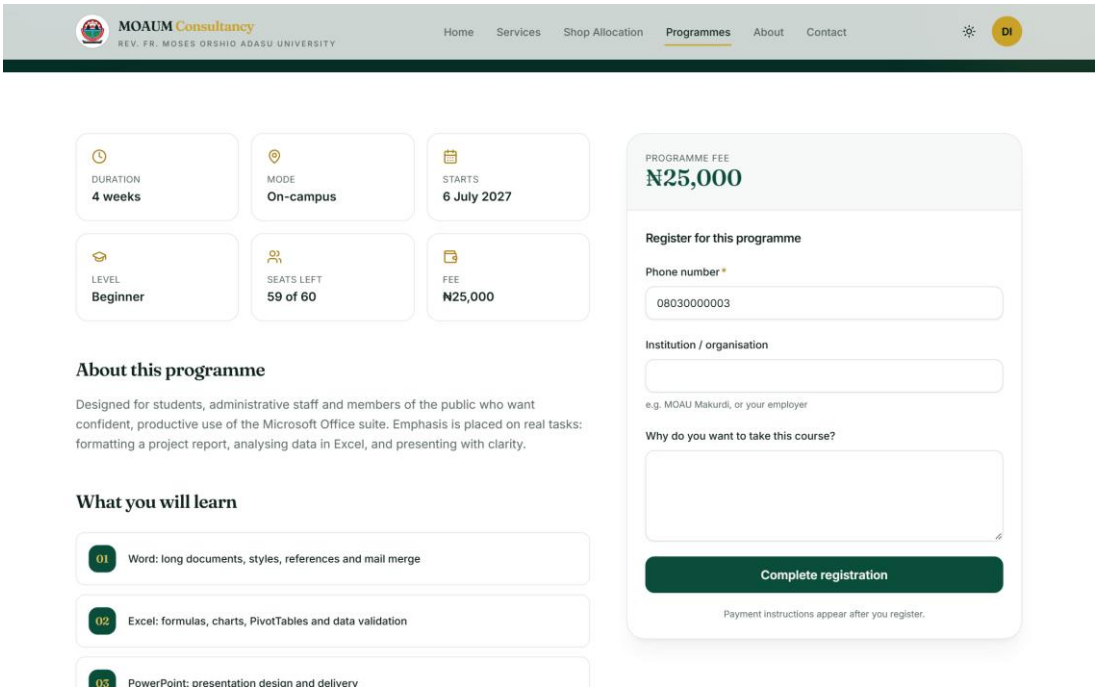



*Figure 4.7: Programme details and registration*

Payment can be made in two ways (Figure 4.8). With **Pay online**, the server starts a Paystack checkout for the exact fee on record and redirects the participant to Paystack's secure payment page; on return, the server verifies the transaction directly with Paystack before marking the registration *Paid* and *Confirmed* (Appendix A, Listing A.7). Alternatively, the participant pays by **bank transfer** and enters the transaction reference, after which staff verify the payment manually.

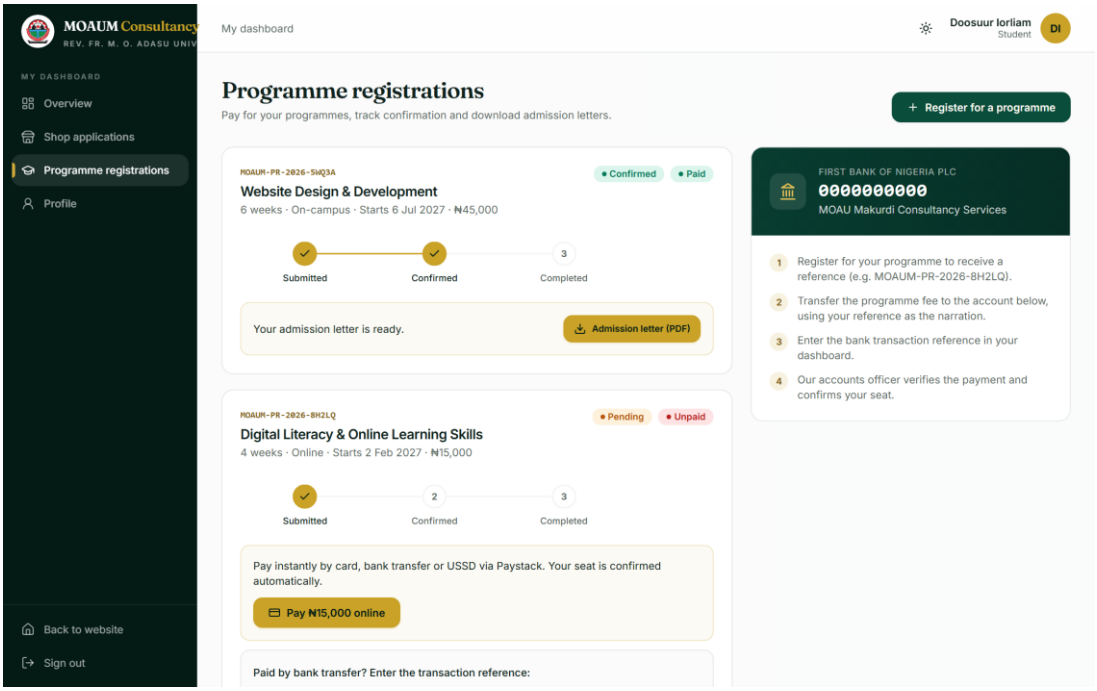



*Figure 4.8: Registrations with online payment and bank-transfer options*

### *4.2.5 Applicant Dashboard*

The dashboard (Figure 4.9) summarises the user's shop applications, registrations, outstanding payments and available letters, with a feed of recent activity. Dedicated pages list each application and registration with its status timeline, the documents submitted and, where applicable, a button to download the official letter (Figure 4.10).

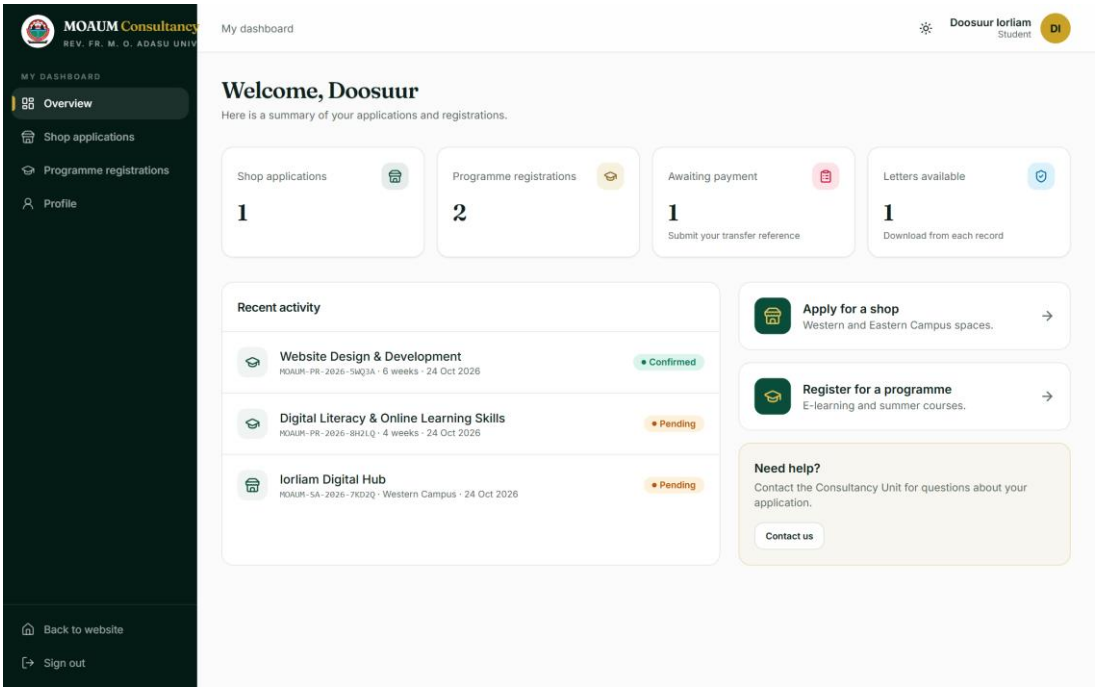



*Figure 4.9: Applicant dashboard*

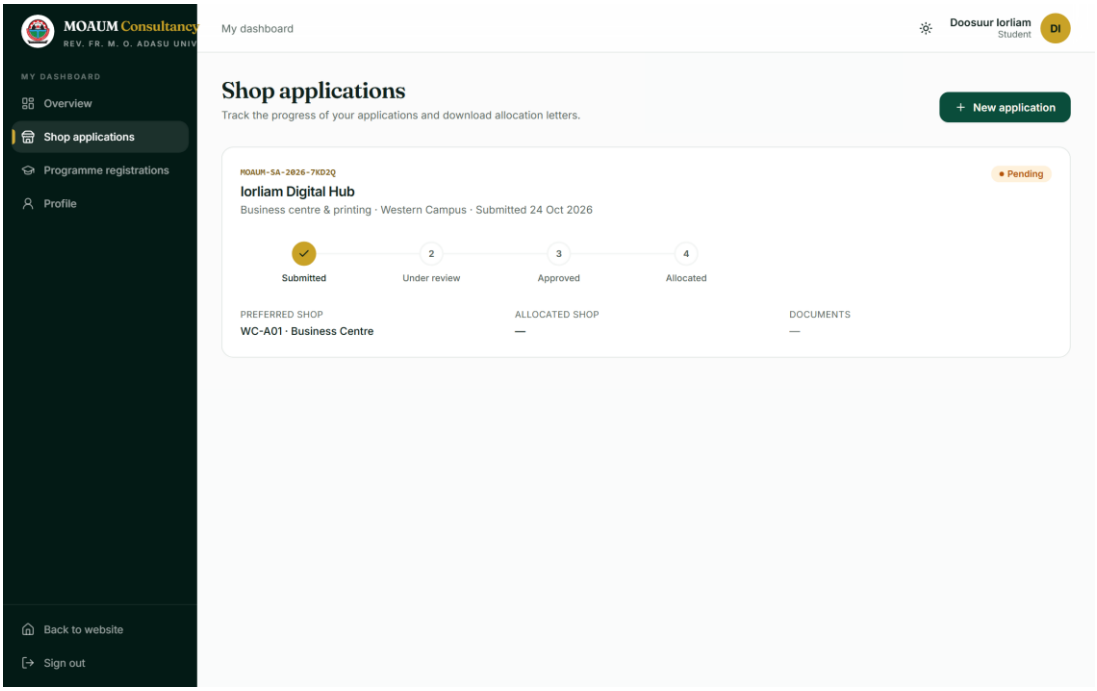



*Figure 4.10: Applicant's shop applications with status timeline*

### *4.2.6 Administrative Console*

Staff and administrators use the admin console. The overview (Figure 4.11) shows applications awaiting review, shop occupancy and annual rent roll, paid programme revenue and registered users, with charts of shops by campus and status,

applications by status and registrations by programme, and buttons to export applications and registrations as CSV files.

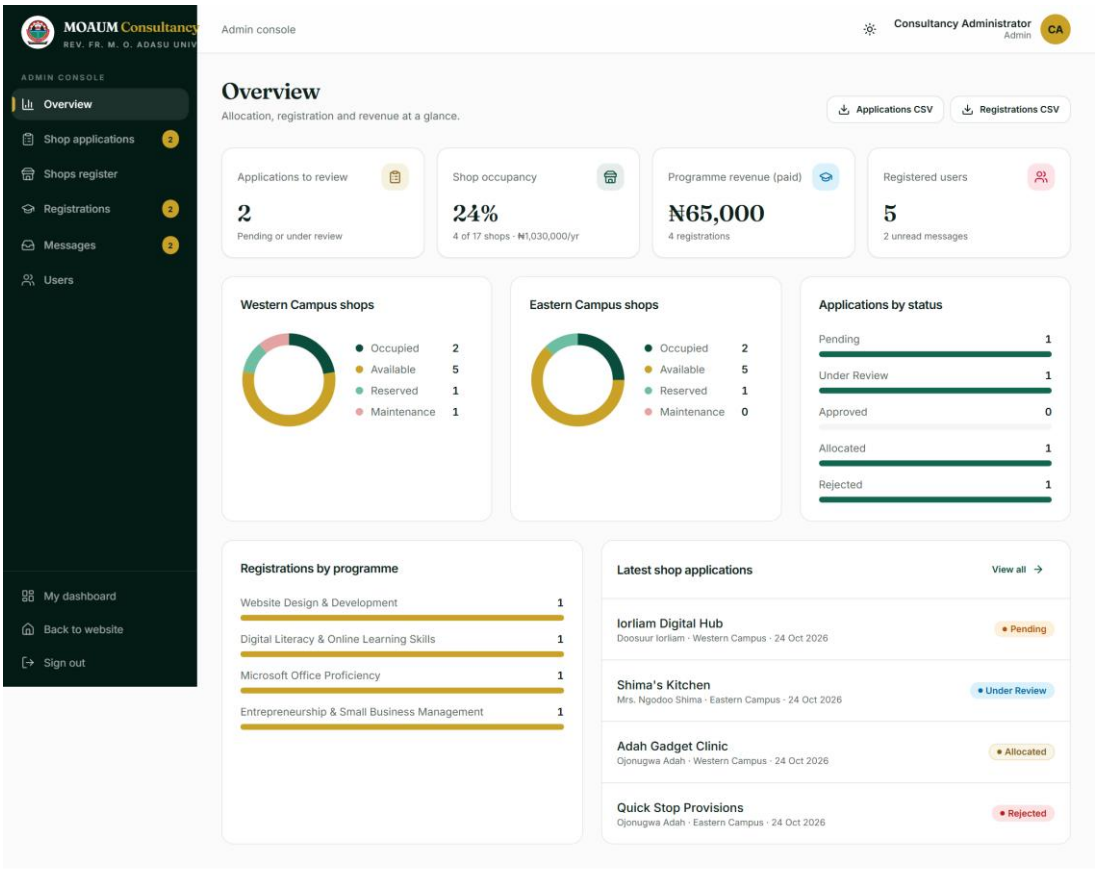



*Figure 4.11: Admin console overview and statistics*

The shop applications page (Figure 4.12) lists all applications with search and filters by campus and status. Opening an application shows the applicant's details, the business description and the uploaded documents, and lets the reviewer record a decision with a note, approve, reject or **allocate** a specific shop (Figure 4.13). Allocation marks the shop *Occupied*, releases any previously allocated shop if the decision changes, and emails the applicant (Appendix A, Listing A.6).

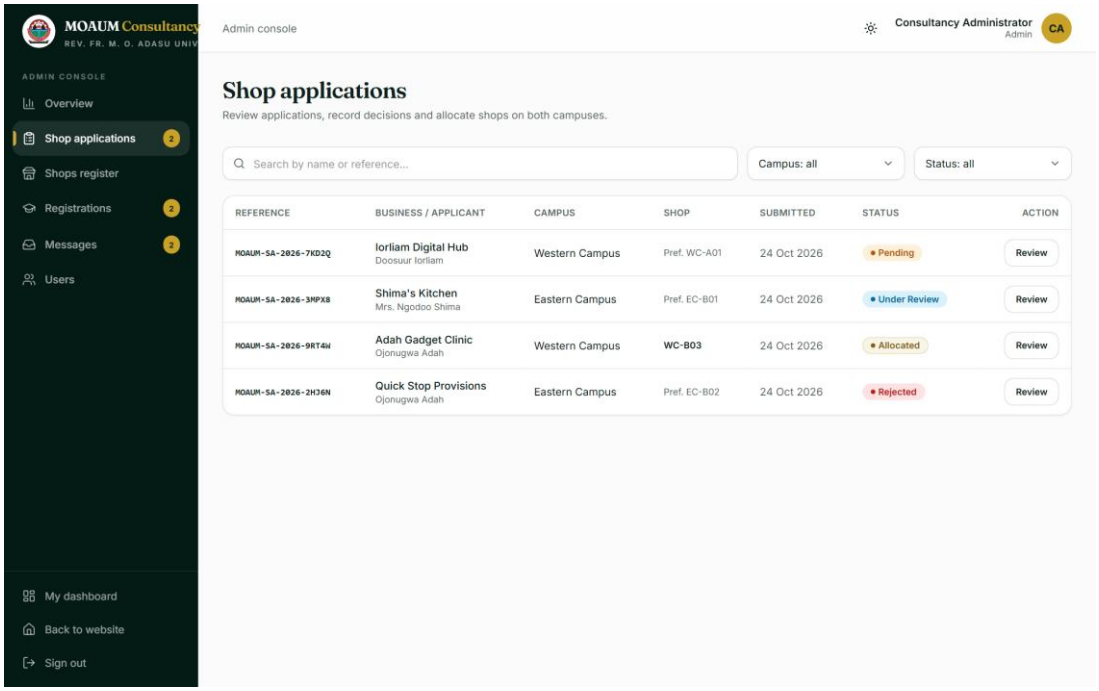



*Figure 4.12: Shop applications awaiting review*

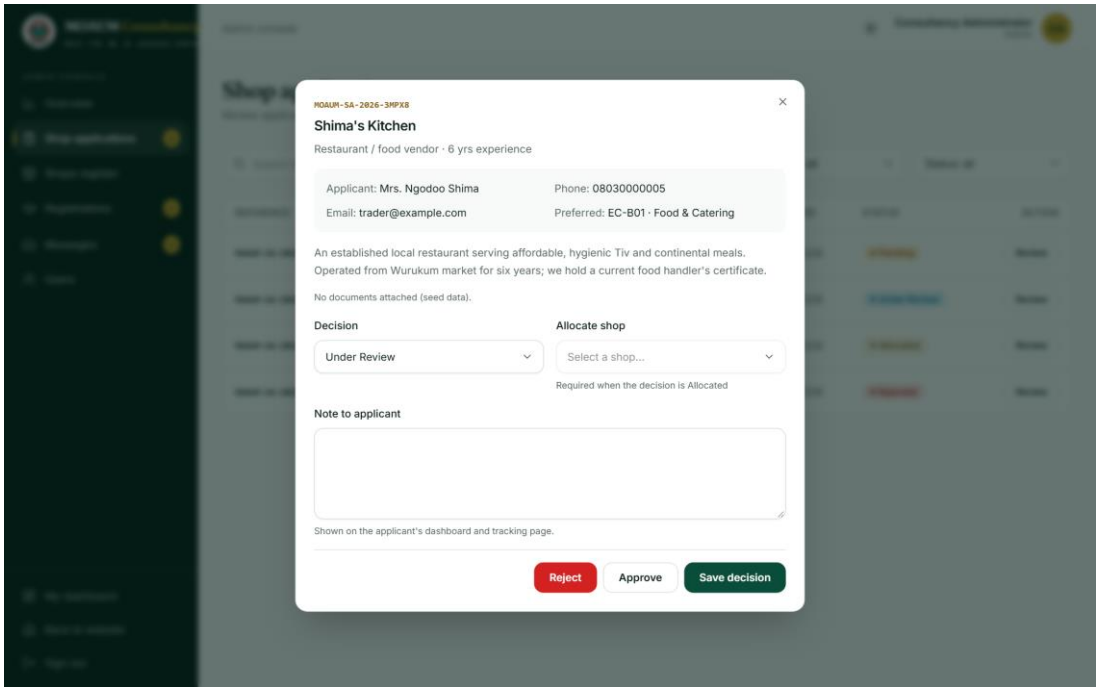



*Figure 4.13: Reviewing an application and allocating a shop*

Further pages allow staff to verify payments and confirm registrations (Figure 4.14), maintain the shops register, read contact enquiries and, for administrators only, change user roles and deactivate accounts (Appendix B).

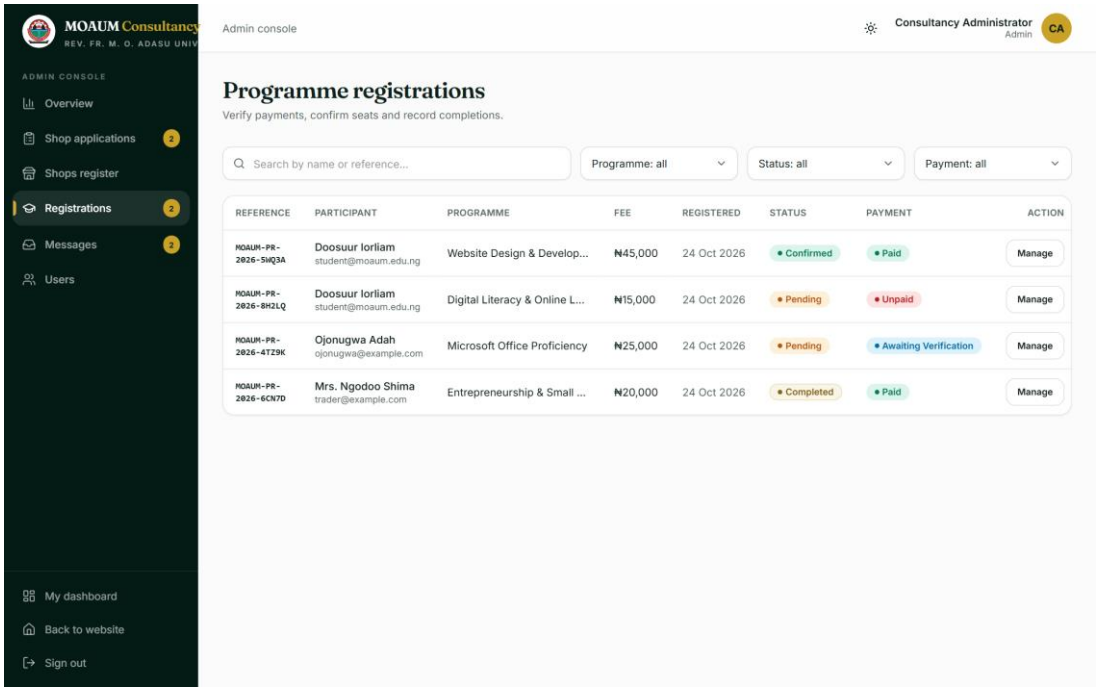



*Figure 4.14: Verifying payments and confirming registrations*

### *4.2.7 Email Notifications*

Email is sent through Nodemailer over SMTP using the Unit's Gmail account. Branded HTML messages are sent when a shop application is received; whenever its status or committee note changes (the allocation email links to the letter); when a registration is received, confirmed or its payment verified; and when an enquiry is submitted (an acknowledgement to the sender and the enquiry to the Unit's inbox, with reply-to set to the sender). Sending never raises an error to the user: if the mail server is unavailable the failure is logged and the original action still succeeds (Appendix A, Listing A.11).

### *4.2.8 Official Letters (PDF)*

Allocation letters and admission letters are generated on demand with pdf-lib as A4 documents carrying the University crest and letterhead, the reference number, the recipient's details, a table of the shop or programme particulars, the signatory and a verification footer (Figure 4.15; Appendix A, Listing A.10). Letters are only available once an application is *Allocated* or a registration is *Confirmed*, and only to the owner

or to staff. Any characters that the standard PDF fonts cannot display, such as the Naira sign or curly quotation marks, are converted first so that unusual input can never cause letter generation to fail.



REV. FR. MOSES ORSHIO ADASU UNIVERSITY, MAKURDI
CONSULTANCY UNIT · MAKURDI, BENUE STATE
"Scientia Liberatio Populorum"

Ref: MOAUM-SA-2026-9RT4W 24 October 2026

Ojonugwa Adah
ojonugwa@example.com
08030000004

LETTER OF SHOP ALLOCATION

Dear Ojonugwa Adah,

Following the review of your application in respect of "Adah Gadget Clinic", I am pleased to inform you that the Shop Allocation Committee of the Consultancy Unit has approved the allocation of a commercial space to you on the Western Campus, subject to the terms below.

This allocation is personal to you and may not be transferred or sub-let. You are required to operate the approved business only, comply with University regulations, display regulated prices and maintain the premises in a clean and safe condition.

| | |
|---|---|
| Shop code | WC-B03 |
| Location | Block B, Faculty of Science axis |
| Category | Retail |
| Floor area | 14 square metres |
| Annual rent | NGN 280,000 |
| Caution deposit (10%) | NGN 28,000 |

Kindly present this letter at the Bursary within fourteen (14) days to make payment and sign the tenancy agreement. Keys will be released upon confirmation of payment. Failure to do so within the stated period may result in the allocation being withdrawn. Congratulations.

Prof. Iorwuese T. Aondoakaa
Director, Consultancy Unit

consultancy@moaum.edu.ng · +234 803 000 0000 · Verify this letter online with reference MOAUM-SA-2026-9RT4W

*Figure 4.15: Generated letter of shop allocation (PDF)*

## 4.3 Testing

Testing was carried out at the end of every increment and again on the deployed system. Four kinds of test were performed:

- **Static checks:** the TypeScript compiler and the Next.js production build were run after every change; both passed without errors for the final version.
- **Functional (black-box) testing:** each use case was exercised end to end in a browser with the seeded demo accounts, for example applying for a shop with documents, allocating it as an administrator and downloading the letter.
- **Security testing:** protected pages and files were requested without signing in, as a student and as staff, to confirm that each role can reach only what it is allowed to; forged payment webhooks were sent to confirm they are rejected.
- **Responsiveness testing:** every public page was measured at a width of 375 pixels to confirm there is no horizontal scrolling.

In addition, the payment-processing logic was tested directly with simulated Paystack transactions covering a failed charge, an underpayment, the wrong currency, an unknown registration, a valid payment and a duplicate notification, and a complete test-mode Paystack checkout was carried out. Table 4.1 summarises the results; the full list of test cases is given in Appendix D.

**Table 4.1: Summary of test results**

| Test area | Cases | Passed | Result |
|---|---|---|---|
| Public pages and navigation | 3 | 3 | Pass |
| Accounts and authentication | 3 | 3 | Pass |
| Shop allocation workflow | 5 | 5 | Pass |
| Programme registration and payment | 7 | 7 | Pass |
| Notifications and reports | 4 | 4 | Pass |
| Security and access control | 6 | 6 | Pass |
| Responsiveness, build and deployment | 4 | 4 | Pass |
| Total | 32 | 32 | All passed |

Testing exposed several defects, all of which were corrected and re-tested before the final release. Table 4.2 lists them; they illustrate the value of testing each increment rather than only at the end.

**Table 4.2: Defects found during testing and their correction**

| Defect found | Correction |
|---|---|
| Main page content stayed invisible until all JavaScript had loaded, a problem on slow connections. | Entrance animations were changed to CSS so that server-rendered content is visible immediately. |
| Submitting a form before the page finished loading placed the entered details (e.g. phone number) in the address bar. | All forms were set to submit with the POST method, so data can never appear in a URL. |
| The programme registration form could remain invisible because of an animation start-up issue. | The animation was configured not to run on first display. |
| Choosing a shop on the Eastern Campus did not pre-select it in the application form. | The campus and shop selectors were made to follow the form state. |
| A name containing characters such as the Naira sign could cause PDF letter generation to fail. | Text is converted to characters supported by the PDF fonts before drawing. |
| A deactivated account kept access until its session token expired. | The session is now re-validated against the database on every request. |
| The sign-in page scrolled sideways when a field was focused; campus tabs overflowed by 2 px on phones. | Decorative elements were clipped and the tabs were laid out in a grid. |
| The first cloud deployments failed because the database connection was not found. | The build step was made to locate the database settings under any name and to report clearly when they are missing. |

## 4.4 Deployment

The production system runs on Vercel and is available at **https://moaum-consultancy.vercel.app**. Because the development configuration uses SQLite and a local folder, three changes prepared the code for cloud hosting: uploaded documents are written to Vercel Blob when its access token is present; a build script switches the database provider to PostgreSQL only on Vercel (Appendix A, Listing A.12); and a production seed loads the shop and programme catalogue and creates a single administrator account from configured settings, never the demonstration accounts.

Deployment was carried out as follows: the repository was imported into Vercel; a Neon PostgreSQL database and a Blob store were added through Vercel's Storage integration; the secrets (session secret, administrator credentials, SMTP and Paystack keys) were entered as environment variables; and the project was deployed.

Thereafter, every push to the main branch on GitHub automatically runs the `vercel-build` script, which prepares the database provider, applies the schema, seeds the catalogue if necessary and builds the site before publishing it (Figure 2.1, delivery pipeline). Vercel only replaces the live version if the build succeeds, so a faulty change cannot take the site offline.

### 4.5 Discussion of Results

All fourteen functional requirements in Table 2.1 were implemented and demonstrated, and the non-functional requirements in Table 2.2 were met: access control was verified for every role, the interface works from phone to desktop, and payments are verified with the provider rather than trusted from the browser. Compared with the existing manual process, an applicant can now see availability, apply with documents, pay and follow progress without visiting the office, while the Unit gains an auditable record of every decision, automatic letters and notifications, and immediate statistics.

# CHAPTER FIVE
# SUMMARY, CONCLUSION AND RECOMMENDATIONS

## 5.1 Summary

The project set out to replace the Consultancy Unit's paper-based processes with a secure web portal. The existing process was analysed, requirements were specified, and the system was modelled with use case and class diagrams. It was then implemented incrementally with Next.js, React, Tailwind CSS, Prisma and NextAuth.js, integrated with Paystack, Gmail SMTP and Vercel Blob, tested functionally and for security and responsiveness, and deployed to Vercel with continuous deployment from GitHub.

## 5.2 Conclusion

The MOAUM Consultancy Portal meets its aim and objectives. It provides the public with clear information on campus shops and programmes; gives applicants an online way to apply, upload documents, pay and track progress; and gives staff and administrators a console for reviewing and allocating applications, verifying payments, generating letters, managing users and monitoring activity. The project demonstrates the value of a disciplined software engineering process: careful modelling clarified the system before coding, incremental delivery produced a working system at every stage, and systematic testing found and removed real defects before release.

## 5.3 Recommendations

1. The Consultancy Unit should supply its confirmed contact details, officers' names and bank account so that the placeholder content can be replaced before the portal is announced.
2. The Unit should complete Paystack business verification and switch to the live payment key, and register the production webhook address.

3. A dedicated transactional email service should replace the Gmail account if monthly email volume grows beyond a few thousand messages.
4. A custom domain under the University's domain (for example consultancy.bsum.edu.ng) should be configured for credibility.
5. Staff should be trained on the admin console, and a periodic review of user roles and accounts should be instituted.

### 5.4 Suggestions for Further Work

- Online booking and payment for facilities such as halls and laboratories.
- Automatic rent reminders and renewal of shop tenancies.
- SMS notifications for applicants without reliable email access.
- Integration with the University's student information system for single sign-on and automatic verification of matriculation numbers.
- An automated test suite (unit and end-to-end tests) run on every change.

# REFERENCES


Anthropic. (2026). *Claude Code* [AI-assisted software development tool]. https://claude.com/claude-code

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. https://agilemanifesto.org

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.

Meta Platforms, Inc. (2026). *React documentation*. https://react.dev

NextAuth.js. (2024). *NextAuth.js documentation* (Version 4). https://next-auth.js.org

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

Paystack. (2026). *Paystack API reference*. https://paystack.com/docs/api/

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Prisma Data, Inc. (2024). *Prisma ORM documentation*. https://www.prisma.io/docs

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Tailwind Labs. (2024). *Tailwind CSS documentation* (Version 3). https://v3.tailwindcss.com/docs

Vercel, Inc. (2026a). *Next.js documentation*. https://nextjs.org/docs

Vercel, Inc. (2026b). *Vercel documentation*. https://vercel.com/docs

Wikipedia contributors. (2026). Benue State University. In *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Benue_State_University

# APPENDIX A: SELECTED SOURCE CODE

This appendix reproduces the most important parts of the source code. Line numbers refer to the files in the project repository (https://github.com/aligodwininalegwu-dotcom/moaum-consultancy).

## A.1 Database Schema

**Listing A.1: Prisma data model** *(prisma/schema.prisma, lines 7–156)*

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(cuid())
  name         String
  email        String   @unique
  passwordHash String
  /// STUDENT | STAFF | ADMIN  (unauthenticated visitors are "PUBLIC")
  role         String   @default("STUDENT")
  phone        String?
  matricNo     String?
  department   String?
  active       Boolean  @default(true)
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  shopApplications ShopApplication[] @relation("Applicant")
  reviewedShopApps ShopApplication[] @relation("Reviewer")
  registrations    Registration[]
}

model Shop {
  id          String   @id @default(cuid())
  /// e.g. WC-A01 (Western Campus, Block A, Unit 01)
  code        String   @unique
  /// WESTERN | EASTERN
  campus      String
  block       String
  location    String
  /// Business Centre | Food & Catering | Retail | Services | Kiosk
  category    String
  sizeSqm     Float
  /// Annual rent in Naira
  annualRent  Int
  /// AVAILABLE | OCCUPIED | RESERVED | MAINTENANCE
  status      String   @default("AVAILABLE")
  /// Comma-separated list, e.g. "Electricity,Water,Burglary-proof"
  features    String   @default("")
  description String   @default("")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  preferredBy ShopApplication[] @relation("PreferredShop")
  allocatedTo ShopApplication[] @relation("AllocatedShop")
}

model ShopApplication {
  id               String   @id @default(cuid())
  /// Public tracking reference, e.g. MOAUM-SA-2026-4F7K2
  reference        String   @unique
  campus           String
  businessName     String
  businessType     String
  description      String
  yearsExperience  Int      @default(0)
```

```
  phone            String
  /// Stored file paths, relative to the uploads directory
  idDocument       String?
  proposalDocument String?
  /// PENDING | UNDER_REVIEW | APPROVED | REJECTED | ALLOCATED
  status           String   @default("PENDING")
  adminNote        String?
  createdAt        DateTime @default(now())
  updatedAt        DateTime @updatedAt

  applicantId     String
  applicant       User    @relation("Applicant", fields: [applicantId], references: [id],
onDelete: Cascade)
  preferredShopId String?
  preferredShop   Shop?   @relation("PreferredShop", fields: [preferredShopId], references:
[id], onDelete: SetNull)
  allocatedShopId String?
  allocatedShop   Shop?   @relation("AllocatedShop", fields: [allocatedShopId], references:
[id], onDelete: SetNull)
  reviewedById    String?
  reviewedBy      User?   @relation("Reviewer", fields: [reviewedById], references: [id],
onDelete: SetNull)

  @@index([status])
  @@index([campus])
}

model Programme {
  id          String   @id @default(cuid())
  slug        String   @unique
  title       String
  /// ELEARNING | SUMMER
  category    String
  summary     String
  description String
  duration    String
  /// Online | On-campus | Hybrid
  mode        String
  level       String
  /// Fee in Naira
  fee         Int
  seats       Int
  startDate   DateTime
  /// JSON-encoded string[] of modules / topics
  modules     String   @default("[]")
  /// Lucide icon name used on course cards
  icon        String   @default("BookOpen")
  featured    Boolean  @default(false)
  active      Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  registrations Registration[]
}

model Registration {
  id               String   @id @default(cuid())
  /// e.g. MOAUM-PR-2026-8H2LQ
  reference        String   @unique
  /// PENDING | CONFIRMED | REJECTED | COMPLETED
  status           String   @default("PENDING")
  /// UNPAID | AWAITING_VERIFICATION | PAID
  paymentStatus    String   @default("UNPAID")
  paymentReference String?
  phone            String
  institution      String?
  motivation       String?
  adminNote        String?
  createdAt        DateTime @default(now())
  updatedAt        DateTime @updatedAt

  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  programmeId String
  programme   Programme @relation(fields: [programmeId], references: [id], onDelete: Cascade)

  @@unique([userId, programmeId])
  @@index([status])
}

model ContactMessage {
  id        String   @id @default(cuid())
  name      String
  email     String
```

```
    phone     String?
    /// General Enquiry | Shop Allocation | Programmes | Career Guidance | Training Workshop |
Facility Booking | Consultancy
    category  String
    subject   String
    message   String
    read      Boolean  @default(false)
    createdAt DateTime @default(now())
  }
```

## A.2 Input Validation

**Listing A.2: Shared validation rules (Zod)** *(src/lib/validations.ts, lines 1–56)*

```
import { z } from "zod";
import { CAMPUSES, CONTACT_CATEGORIES } from "@/lib/constants";

const phone = z
  .string()
  .trim()
  .regex(/^(\+?234|0)[789][01]\d{8}$/, "Enter a valid Nigerian phone number, e.g.
08031234567");

export const loginSchema = z.object({
  email: z.string().trim().email("Enter a valid email address"),
  password: z.string().min(1, "Password is required"),
});

export const signupSchema = z
  .object({
    name: z.string().trim().min(3, "Enter your full name"),
    email: z.string().trim().email("Enter a valid email address"),
    phone,
    matricNo: z.string().trim().max(30).optional().or(z.literal("")),
    department: z.string().trim().max(80).optional().or(z.literal("")),
    password: z
      .string()
      .min(8, "At least 8 characters")
      .regex(/[A-Za-z]/, "Include at least one letter")
      .regex(/\d/, "Include at least one number"),
    confirmPassword: z.string(),
  })
  .refine((d) => d.password === d.confirmPassword, {
    path: ["confirmPassword"],
    message: "Passwords do not match",
  });
export type SignupInput = z.infer<typeof signupSchema>;

export const shopApplicationSchema = z.object({
  campus: z.enum(CAMPUSES),
  preferredShopId: z.string().optional().or(z.literal("")),
  businessName: z.string().trim().min(3, "Enter your business name").max(100),
  businessType: z.string().trim().min(3, "Describe the type of business").max(80),
  phone,
  yearsExperience: z.coerce.number().int().min(0).max(60),
  description: z
    .string()
    .trim()
    .min(60, "Give at least a short paragraph (60+ characters) about the business")
    .max(3000),
  agree: z.literal(true, { errorMap: () => ({ message: "You must accept the allocation
guidelines" }) }),
});
export type ShopApplicationInput = z.infer<typeof shopApplicationSchema>;

export const registrationSchema = z.object({
  programmeId: z.string().min(1),
  phone,
  institution: z.string().trim().max(120).optional().or(z.literal("")),
  motivation: z.string().trim().max(1000).optional().or(z.literal("")),
});
export type RegistrationInput = z.infer<typeof registrationSchema>;
```

## A.3 Authentication and Role Checks

**Listing A.3: NextAuth configuration and server-side guards** *(src/lib/auth.ts, lines 1–85)*

```
import type { NextAuthOptions } from "next-auth";
import { getServerSession } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcryptjs";
import { redirect } from "next/navigation";
import { prisma } from "@/lib/prisma";
import { CONSOLE_ROLES, type Role } from "@/lib/constants";
import { loginSchema } from "@/lib/validations";

export const authOptions: NextAuthOptions = {
  session: { strategy: "jwt", maxAge: 60 * 60 * 24 * 7 },
  pages: { signIn: "/login" },
  providers: [
    CredentialsProvider({
      name: "Email and password",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials) {
        const parsed = loginSchema.safeParse(credentials);
        if (!parsed.success) return null;

        const user = await prisma.user.findUnique({
          where: { email: parsed.data.email.toLowerCase() },
        });
        if (!user || !user.active) return null;

        const valid = await bcrypt.compare(parsed.data.password, user.passwordHash);
        if (!valid) return null;

        return { id: user.id, name: user.name, email: user.email, role: user.role as Role };
      },
    }),
  ],
  callbacks: {
    async jwt({ token, user, trigger }) {
      if (user) {
        token.id = user.id;
        token.role = (user as { role: Role }).role;
        return token;
      }
      // On every later session read, re-check the account so role changes and
      // deactivations take effect immediately instead of when the JWT expires.
      // (Runs only in Node route handlers / server components — the edge
      // middleware just decodes the token.)
      if (token.id) {
        const fresh = await prisma.user.findUnique({ where: { id: token.id }, select: { name:
true, role: true, active: true } });
        if (!fresh || !fresh.active) return { sub: token.sub }; // signed out: no id/role
        token.name = fresh.name;
        token.role = fresh.role as Role;
      }
      void trigger; // useSession().update() lands here too and gets the fresh values above
      return token;
    },
    async session({ session, token }) {
      if (!token.id) return { expires: session.expires };
      if (session.user) {
        session.user.id = token.id;
        session.user.role = token.role as Role;
      }
      return session;
    },
  },
};

/** The current session, or null if signed out or the account is gone/deactivated. */
export async function getSession() {
  const session = await getServerSession(authOptions);
  return session?.user?.id ? session : null;
}

/** Server-side guard: returns the signed-in user or redirects to login. */
export async function requireUser(callbackUrl = "/dashboard") {
  const session = await getSession();
  if (!session) redirect(`/login?callbackUrl=${encodeURIComponent(callbackUrl)}`);
  return session.user;
```

```
78  }
79
80  /** Server-side guard for the admin console (staff + admins). */
81  export async function requireConsole(roles: Role[] = CONSOLE_ROLES) {
82    const user = await requireUser("/admin");
83    if (!roles.includes(user.role)) redirect("/dashboard");
84    return user;
85  }
```

## A.4 Route Protection

**Listing A.4: Middleware protecting the dashboard and console** *(src/middleware.ts, lines 1–7)*

```
1  import { withAuth } from "next-auth/middleware";
2
3  // Any signed-in user may enter /dashboard and /admin; role checks for the
4  // admin console happen in its layout so students are redirected, not bounced to login.
5  export default withAuth({ pages: { signIn: "/login" } });
6
7  export const config = { matcher: ["/dashboard/:path*", "/admin/:path*"] };
```

## A.5 Submitting a Shop Application

**Listing A.5: Server Action for shop applications** *(src/server/actions.ts, lines 84–153)*

```
 84  export async function submitShopApplication(formData: FormData): Promise<ActionResult<{
reference: string }>> {
 85    const session = await getSession();
 86    if (!session?.user) return { ok: false, error: "Please sign in to apply for a shop." };
 87
 88    const parsed = shopApplicationSchema.safeParse({
 89      campus: formData.get("campus"),
 90      preferredShopId: formData.get("preferredShopId") ?? "",
 91      businessName: formData.get("businessName"),
 92      businessType: formData.get("businessType"),
 93      phone: formData.get("phone"),
 94      yearsExperience: formData.get("yearsExperience"),
 95      description: formData.get("description"),
 96      agree: formData.get("agree") === "true",
 97    });
 98    if (!parsed.success) return { ok: false, error: "Please correct the highlighted fields.",
fieldErrors: parsed.error.flatten().fieldErrors };
 99
100    const idDocument = formData.get("idDocument");
101    const proposalDocument = formData.get("proposalDocument");
102    const fileError = validateUpload(idDocument, "Means of identification") ??
validateUpload(proposalDocument, "Business proposal");
103    if (fileError) return { ok: false, error: fileError };
104
105    const data = parsed.data;
106
107    const active = await prisma.shopApplication.findFirst({
108      where: { applicantId: session.user.id, campus: data.campus, status: { in: ["PENDING",
"UNDER_REVIEW", "APPROVED"] } },
109    });
110    if (active) {
111      return { ok: false, error: `You already have an active application (${active.reference})
for this campus.` };
112    }
113
114    if (data.preferredShopId) {
115      const shop = await prisma.shop.findUnique({ where: { id: data.preferredShopId } });
116      if (!shop || shop.campus !== data.campus || shop.status !== "AVAILABLE") {
117        return { ok: false, error: "The selected shop is no longer available. Please choose
another." };
118      }
119    }
120
121    const reference = makeReference("SA");
122    const [idPath, proposalPath] = await Promise.all([
123      saveUpload(idDocument as File, `${reference}-id`),
124      saveUpload(proposalDocument as File, `${reference}-proposal`),
125    ]);
126
127    await prisma.shopApplication.create({
128      data: {
```

```
        reference,
        campus: data.campus,
        businessName: data.businessName,
        businessType: data.businessType,
        description: data.description,
        yearsExperience: data.yearsExperience,
        phone: data.phone,
        idDocument: idPath,
        proposalDocument: proposalPath,
        applicantId: session.user.id,
        preferredShopId: data.preferredShopId || null,
      },
    });

    revalidatePath("/dashboard");
    revalidatePath("/admin/shop-applications");
    if (session.user.email) {
      await mail.shopApplicationReceived(session.user.email, session.user.name ?? "Applicant", {
        reference,
        businessName: data.businessName,
        campus: CAMPUS_LABEL[data.campus as Campus],
      });
    }
    return { ok: true, message: "Application submitted successfully.", data: { reference } };
  }
```

## A.6 Reviewing and Allocating

**Listing A.6: Admin Server Action to review and allocate** *(src/server/admin-actions.ts, lines 40–93)*

```
40  export async function reviewShopApplication(input: z.infer<typeof reviewSchema>):
Promise<ActionResult> {
41    const user = await guard();
42    if (!user) return { ok: false, error: "You are not authorised to do this." };
43    const parsed = reviewSchema.safeParse(input);
44    if (!parsed.success) return { ok: false, error: "Invalid review data." };
45    const { id, status, allocatedShopId, adminNote } = parsed.data;
46
47    const app = await prisma.shopApplication.findUnique({ where: { id }, include: { applicant: {
select: { name: true, email: true } } } });
48    if (!app) return { ok: false, error: "Application not found." };
49    let shopCode: string | undefined;
50
51    if (status === "ALLOCATED") {
52      if (!allocatedShopId) return { ok: false, error: "Choose a shop to allocate." };
53      const shop = await prisma.shop.findUnique({ where: { id: allocatedShopId } });
54      if (!shop || shop.campus !== app.campus) return { ok: false, error: "Choose a shop on the
applicant's campus." };
55      const freeOrSame = shop.status === "AVAILABLE" || shop.status === "RESERVED" ||
app.allocatedShopId === shop.id;
56      if (!freeOrSame) return { ok: false, error: `Shop ${shop.code} is
${shop.status.toLowerCase()}.` };
57      shopCode = shop.code;
58
59      await prisma.$transaction([
60        // Release a previously allocated shop if the allocation is being changed.
61        ...(app.allocatedShopId && app.allocatedShopId !== shop.id
62          ? [prisma.shop.update({ where: { id: app.allocatedShopId }, data: { status: "AVAILABLE"
} })]
63          : []),
64        prisma.shop.update({ where: { id: shop.id }, data: { status: "OCCUPIED" } }),
65        prisma.shopApplication.update({
66          where: { id },
67          data: { status, allocatedShopId: shop.id, adminNote: adminNote || null, reviewedById:
user.id },
68        }),
69      ]);
70    } else {
71      await prisma.$transaction([
72        ...(app.allocatedShopId ? [prisma.shop.update({ where: { id: app.allocatedShopId }, data:
{ status: "AVAILABLE" } })] : []),
73        prisma.shopApplication.update({
74          where: { id },
75          data: { status, allocatedShopId: null, adminNote: adminNote || null, reviewedById:
user.id },
76        }),
77      ]);
78    }
```

```
79
80    refresh();
81    // Tell the applicant only when something they can see actually changed.
82    const changed = status !== app.status || (adminNote || null) !== app.adminNote || (status ===
"ALLOCATED" && allocatedShopId !== app.allocatedShopId);
83    if (changed) {
84      await mail.shopApplicationUpdated(app.applicant.email, app.applicant.name, {
85        reference: app.reference,
86        businessName: app.businessName,
87        status,
88        note: adminNote,
89        shopCode,
90      });
91    }
92    return { ok: true, message: `Application ${app.reference} updated.` };
93  }
```

## A.7 Verifying Online Payments

### Listing A.7: Paystack webhook signature check and payment verification

*(src/server/paystack.ts, lines 58–105)*

```
58  /** Checks the x-paystack-signature header (HMAC-SHA512 of the raw body). */
59  export function isValidSignature(rawBody: string, signature: string | null) {
60    if (!signature || !paystackEnabled()) return false;
61    const expected = createHmac("sha512", secret()).update(rawBody).digest("hex");
62    const a = Buffer.from(expected);
63    const b = Buffer.from(signature);
64    return a.length === b.length && timingSafeEqual(a, b);
65  }
66
67  export type ApplyResult = "paid" | "already-paid" | "not-successful" | "mismatch" | "not-
found";
68
69  /**
70   * Marks a registration paid from a transaction that has been verified with
71   * Paystack. Safe to call twice (callback + webhook both arrive): the second
72   * call is a no-op.
73   */
74  export async function applyVerifiedPayment(tx: PaystackTransaction): Promise<ApplyResult> {
75    if (tx.status !== "success") return "not-successful";
76    const meta = typeof tx.metadata === "string" ? safeJson(tx.metadata) : tx.metadata;
77    const registrationId = meta?.registrationId;
78    if (!registrationId) return "not-found";
79
80    const reg = await prisma.registration.findUnique({
81      where: { id: registrationId },
82      include: { programme: true, user: { select: { name: true, email: true } } },
83    });
84    if (!reg) return "not-found";
85    if (reg.paymentStatus === "PAID") return "already-paid";
86    // Never trust the amount from the browser: compare with the fee on record.
87    if (tx.currency !== "NGN" || tx.amount < reg.programme.fee * 100) return "mismatch";
88
89    const updated = await prisma.registration.update({
90      where: { id: reg.id },
91      data: {
92        paymentStatus: "PAID",
93        paymentReference: `PAYSTACK:${tx.reference}`,
94        status: reg.status === "PENDING" ? "CONFIRMED" : reg.status,
95      },
96    });
97    await mail.registrationUpdated(reg.user.email, reg.user.name, {
98      reference: reg.reference,
99      programme: reg.programme.title,
100     status: updated.status,
101     paymentStatus: updated.paymentStatus,
102     note: "Payment received online via Paystack. Thank you.",
103   });
104   return "paid";
105 }
```

## A.8 Document Storage

**Listing A.8: Validating and saving uploaded documents** *(src/server/uploads.ts, lines 35–50)*

```
35  /** Saves an uploaded file and returns the value to store in the database. */
36  export async function saveUpload(file: File, prefix: string) {
37    const name = `${prefix}-${randomUUID()}.${EXT[file.type] ?? "bin"}`;
38
39    if (useBlob()) {
40      const { put } = await import("@vercel/blob");
41      // The random UUID plus Blob's own random suffix make the URL unguessable;
42      // it is never sent to the browser (downloads are proxied by /api/files).
43      const blob = await put(`applications/${name}`, file, { access: "public", contentType:
file.type, addRandomSuffix: true, token: blobToken() });
44      return blob.url;
45    }
46
47    await mkdir(UPLOAD_DIR, { recursive: true });
48    await writeFile(path.join(UPLOAD_DIR, name), Buffer.from(await file.arrayBuffer()));
49    return name;
50  }
```

**Listing A.9: Reading documents back (local disk or Vercel Blob)** *(src/server/uploads.ts, lines 52–72)*

```
52  export async function readUpload(stored: string) {
53    if (stored.startsWith("https://")) {
54      // Only fetch from Vercel Blob, never from an arbitrary stored URL.
55      const url = new URL(stored);
56      if (!url.hostname.endsWith(".blob.vercel-storage.com")) return null;
57      const res = await fetch(url, { cache: "no-store" });
58      if (!res.ok) return null;
59      return { data: Buffer.from(await res.arrayBuffer()), type: res.headers.get("content-type")
?? "application/octet-stream" };
60    }
61
62    // Stored names are generated by saveUpload; reject anything path-like.
63    if (!/^[\w-]+\.(pdf|jpg|png)$/.test(stored)) return null;
64    try {
65      const data = await readFile(path.join(UPLOAD_DIR, stored));
66      const ext = stored.split(".").pop()!;
67      const type = Object.entries(EXT).find(([, e]) => e === ext)?.[0] ?? "application/octet-
stream";
68      return { data, type };
69    } catch {
70      return null;
71    }
72  }
```

## A.9 Letter Generation

**Listing A.10: Building the PDF letter and letterhead (excerpt)** *(src/server/letters.ts, lines 77–110)*

```
77  /** Generates an official A4 letter (allocation / admission) as PDF bytes. */
78  export async function buildLetter(rawInput: LetterInput) {
79    const input = sanitize(rawInput);
80    const pdf = await PDFDocument.create();
81    pdf.setTitle(`${input.title} — ${input.reference}`);
82    pdf.setAuthor(`${site.unit}, ${site.university}`);
83    const page = pdf.addPage([595.28, 841.89]);
84    const { width, height } = page.getSize();
85    const serif = await pdf.embedFont(StandardFonts.TimesRoman);
86    const serifBold = await pdf.embedFont(StandardFonts.TimesRomanBold);
87    const serifItalic = await pdf.embedFont(StandardFonts.TimesRomanItalic);
88    const sans = await pdf.embedFont(StandardFonts.Helvetica);
89    const sansBold = await pdf.embedFont(StandardFonts.HelveticaBold);
90    const margin = 56;
91    const contentWidth = width - margin * 2;
92
93    // Letterhead
94    page.drawRectangle({ x: 0, y: height - 118, width, height: 118, color: GREEN });
95    page.drawRectangle({ x: 0, y: height - 122, width, height: 4, color: GOLD });
96    const logo = await pdf.embedPng(await loadLogo());
97    page.drawImage(logo, { x: margin - 6, y: height - 104, width: 88, height: 88 });
```

```
    page.drawText(site.university.toUpperCase(), { x: margin + 94, y: height - 52, size: 13, font: serifBold, color: rgb(1, 1, 1) });
    page.drawText(`${site.unit.toUpperCase()} · MAKURDI, BENUE STATE`, { x: margin + 94, y: height - 70, size: 9, font: sansBold, color: GOLD });
    page.drawText(`"${site.motto}"`, { x: margin + 94, y: height - 88, size: 9.5, font: serifItalic, color: rgb(0.85, 0.9, 0.87) });

    let y = height - 160;
    page.drawText(`Ref: ${input.reference}`, { x: margin, y, size: 10, font: sansBold, color: INK });
    const dateText = input.date.toLocaleDateString("en-GB", { day: "numeric", month: "long", year: "numeric" });
    page.drawText(dateText, { x: width - margin - sans.widthOfTextAtSize(dateText, 10), y, size: 10, font: sans, color: INK });

    y -= 34;
    for (const line of [input.recipient.name, input.recipient.email, input.recipient.phone ?? ""].filter(Boolean)) {
      page.drawText(line, { x: margin, y, size: 11, font: line === input.recipient.name ? serifBold : serif, color: INK });
      y -= 15;
```

## A.10 Sending Email

**Listing A.11: Fault-tolerant email sending** *(src/server/mail.ts, lines 61–76)*

```
/**
 * Sends an email without ever throwing: a mail outage must not undo or fail
 * the application, registration or review that triggered it.
 */
export async function sendMail({ to, subject, html, replyTo }: Mail) {
  const t = getTransporter();
  if (!t) {
    console.info(`[mail:dev] To: ${to} | Subject: ${subject} (set SMTP_HOST to send real email)`);
    return;
  }
  try {
    await t.sendMail({ from: FROM, to, subject, html, replyTo: replyTo ?? UNIT_INBOX });
  } catch (err) {
    console.error(`[mail] Failed to send "${subject}" to ${to}:`, err);
  }
}
```

## A.11 Deployment Build Step

**Listing A.12: Switching to PostgreSQL on Vercel** *(scripts/prepare-postgres.mjs, lines 1–60)*

```
/**
 * Vercel build step: switch the Prisma datasource from SQLite (local
 * development) to PostgreSQL (Neon) before `prisma generate`.
 *
 * Vercel's Neon integration may name the connection settings with a prefix
 * (e.g. STORAGE_DATABASE_URL) depending on how it was connected, so this
 * finds the right variables by name and points the schema at them.
 *
 * It edits prisma/schema.prisma in the build machine's copy only, and
 * refuses to run outside Vercel unless --force is passed.
 */
import { readFileSync, writeFileSync } from "node:fs";

if (!process.env.VERCEL && !process.argv.includes("--force")) {
  console.log("[prepare-postgres] Not on Vercel; leaving prisma/schema.prisma unchanged.");
  process.exit(0);
}

const isPostgres = (name) => /^postgres(ql)?:\/\//.test(process.env[name] ?? "");
const names = Object.keys(process.env);

// Pooled URL for the app; preferred names first, then any prefixed variant.
const pooledCandidates = [
  "DATABASE_URL",
  "POSTGRES_PRISMA_URL",
  "POSTGRES_URL",
  ...names.filter((n) => /(^|_)DATABASE_URL$/.test(n)),
  ...names.filter((n) => /(^|_)POSTGRES_PRISMA_URL$/.test(n)),
```

```
    ...names.filter((n) => /(^|_)POSTGRES_URL$/.test(n)),
  ];
  // Direct (non-pooled) URL for schema changes.
  const directCandidates = [
    "DATABASE_URL_UNPOOLED",
    "POSTGRES_URL_NON_POOLING",
    ...names.filter((n) => /(^|_)DATABASE_URL_UNPOOLED$/.test(n)),
    ...names.filter((n) => /(^|_)POSTGRES_URL_NON_POOLING$/.test(n)),
  ];

  const urlVar = pooledCandidates.find(isPostgres);
  const directVar = directCandidates.find(isPostgres);

  if (!urlVar) {
    const related = names.filter((n) => /DATABASE|POSTGRES|PG|NEON/i.test(n)).sort();
    console.error("[prepare-postgres] No PostgreSQL connection URL found for this deployment.");
    console.error(`[prepare-postgres] Database-related variables present: ${related.length ?
related.join(", ") : "(none)"}`);
    console.error("[prepare-postgres] Fix: Vercel → Storage → your Neon database → Connect
Project → tick Production (and Preview), then redeploy.");
    process.exit(1);
  }

  const file = "prisma/schema.prisma";
  const schema = readFileSync(file, "utf8");
  const datasource = /datasource db \{[\s\S]*?\}/;
  if (!datasource.test(schema)) throw new Error("datasource block not found in schema.prisma");

  const updated = schema.replace(
    datasource,
    `datasource db {\n  provider = "postgresql"\n  url      = env("${urlVar}")${directVar ? `\n
directUrl = env("${directVar}")` : ""}\n}`,
  );
  writeFileSync(file, updated);
  console.log(`[prepare-postgres] Using PostgreSQL from ${urlVar}${directVar ? ` (schema changes
via ${directVar})` : ""}.`);
```

**Listing A.13: Production seed (catalogue and first administrator)** *(prisma/seed-production.ts, lines 1–58)*

```
/**
 * Production seed — runs on every Vercel deploy and is safe to repeat.
 *
 * - Loads the shop and programme catalogue only if those tables are empty,
 *   so edits made later in the admin console are never overwritten.
 * - Creates ONE administrator from ADMIN_EMAIL / ADMIN_PASSWORD if no user
 *   with that email exists yet. It never creates the demo accounts.
 */
import { PrismaClient } from "@prisma/client";
import bcrypt from "bcryptjs";
import { programmes, shops } from "./catalog";

const prisma = new PrismaClient();

async function main() {
  if ((await prisma.shop.count()) === 0) {
    await prisma.shop.createMany({ data: shops.map((s) => ({ ...s, features:
s.features.join(",") })) });
    console.log(`[seed] Added ${shops.length} shops.`);
  }

  if ((await prisma.programme.count()) === 0) {
    await prisma.programme.createMany({
      data: programmes.map((p) => ({
        ...p,
        startDate: new Date(p.startDate),
        modules: JSON.stringify(p.modules),
        featured: p.featured ?? false,
      })),
    });
    console.log(`[seed] Added ${programmes.length} programmes.`);
  }

  const email = process.env.ADMIN_EMAIL?.trim().toLowerCase();
  const password = process.env.ADMIN_PASSWORD;
  if (!email || !password) {
    console.log("[seed] ADMIN_EMAIL / ADMIN_PASSWORD not set; skipping admin account.");
  } else if (password.length < 10) {
    throw new Error("ADMIN_PASSWORD must be at least 10 characters.");
  } else if (!(await prisma.user.findUnique({ where: { email } }))) {
    await prisma.user.create({
```

```
      data: {
        name: process.env.ADMIN_NAME?.trim() || "Consultancy Administrator",
        email,
        passwordHash: await bcrypt.hash(password, 12),
        role: "ADMIN",
        department: "Consultancy Unit",
      },
    });
    console.log(`[seed] Created administrator ${email}.`);
  }
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());
```

# APPENDIX B: ADDITIONAL SCREENSHOTS

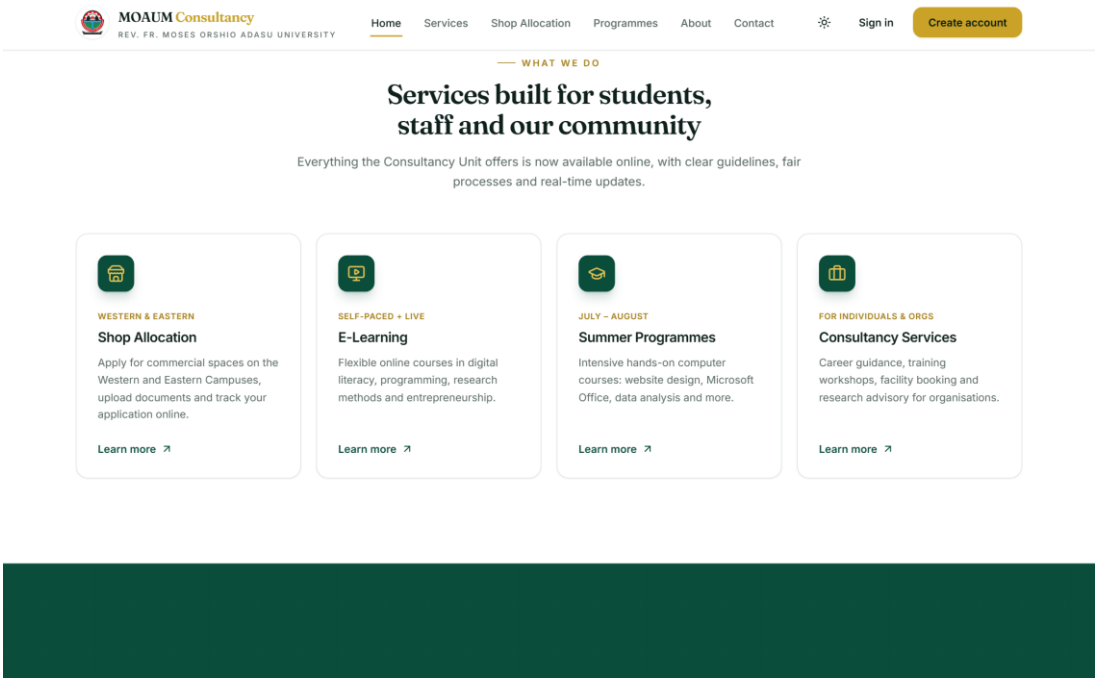



*Figure B.1: Home page – services overview*

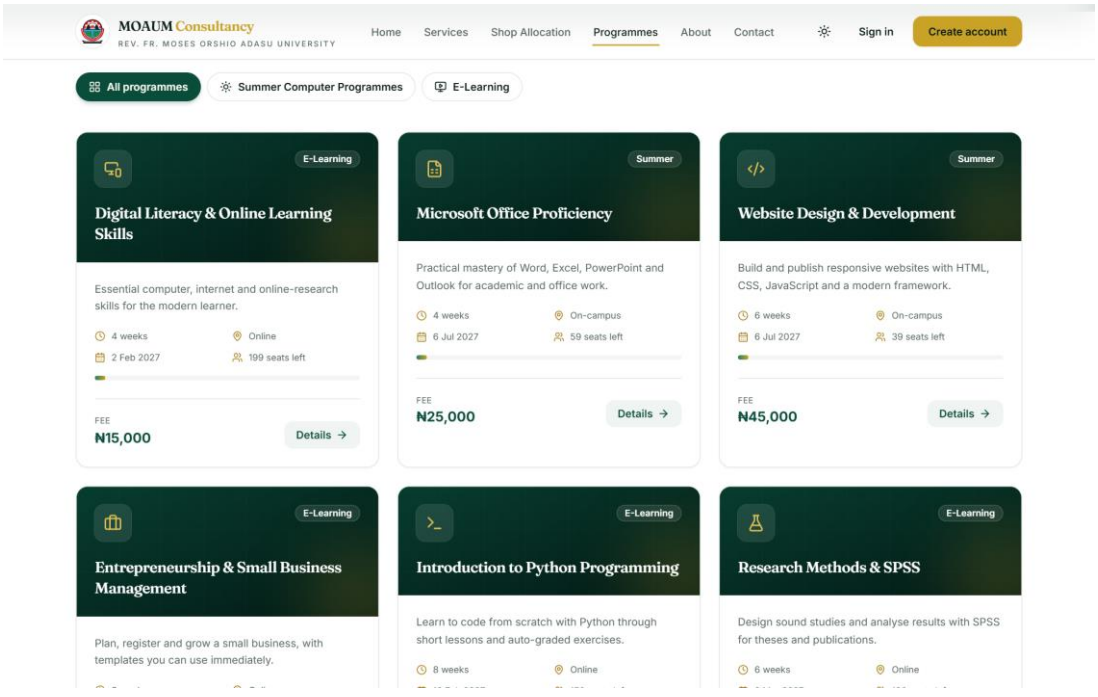



*Figure B.2: Programmes catalogue*

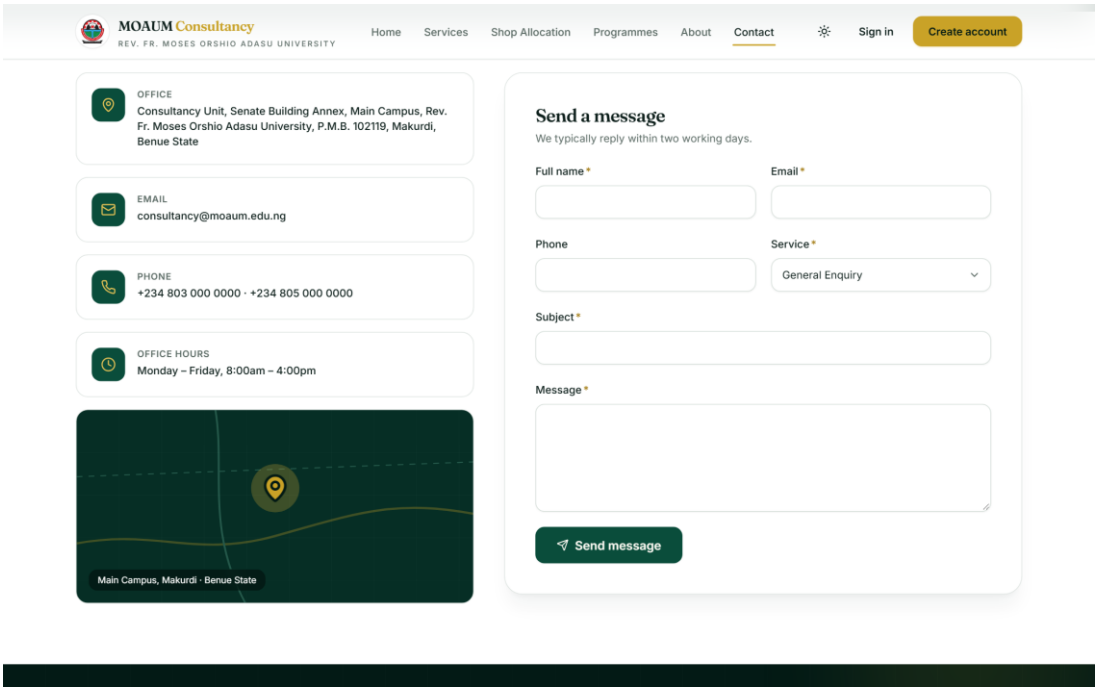



*Figure B.3: Contact page and enquiry form*

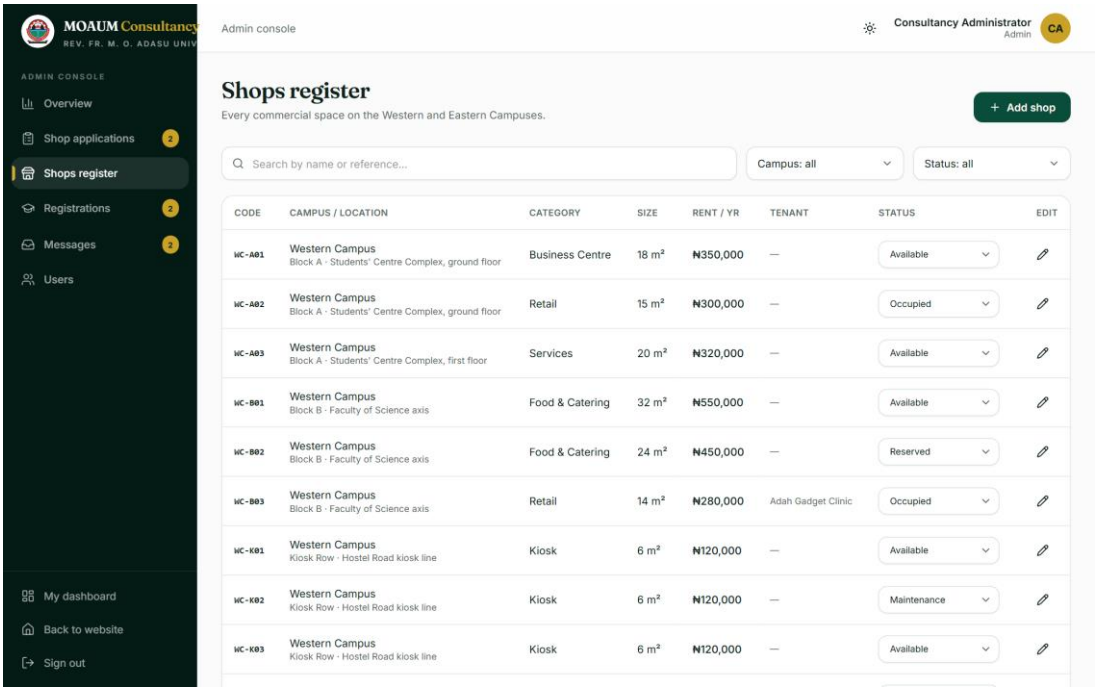



*Figure B.4: Admin console – shops register*

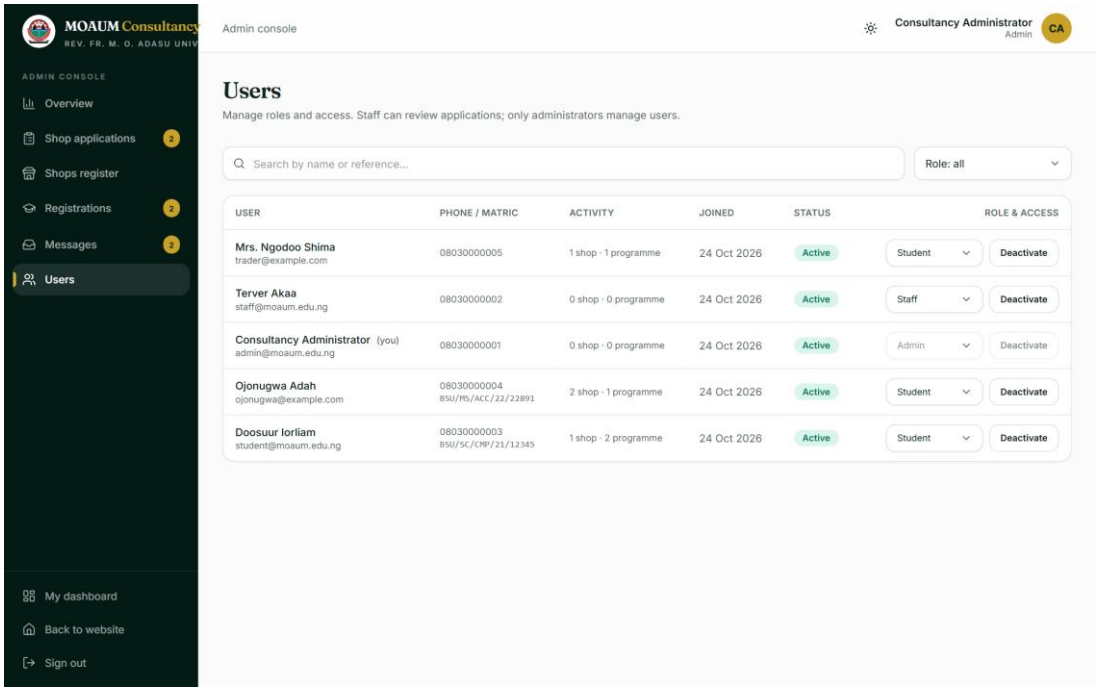



*Figure B.5: Admin console – user management (administrators only)*

# APPENDIX C: SAMPLE GENERATED LETTER

Figure C.1 shows an admission letter generated by the system for a confirmed programme registration (demonstration data). An allocation letter is shown in Figure 4.15.



REV. FR. MOSES ORSHIO ADASU UNIVERSITY, MAKURDI
CONSULTANCY UNIT · MAKURDI, BENUE STATE
*"Scientia Liberatio Populorum"*

**Ref: MOAUM-PR-2026-5WQ3A** 24 October 2026

**Doosuur Iorliam**
student@moaum.edu.ng
08030000003

**LETTER OF ADMISSION**

Dear Doosuur Iorliam,

On behalf of the Consultancy Unit, I am pleased to offer you admission into the Summer Computer Programme: "Website Design & Development". Your registration has been confirmed and your payment verified.

Participants are expected to attend at least 75% of sessions and complete all assessments to qualify for the Certificate of Completion. Please arrive with this letter and a valid means of identification on the first day.

| | |
|---|---|
| Programme | Website Design & Development |
| Duration | 6 weeks |
| Mode of delivery | On-campus |
| Commencement | 6 July 2027 |
| Programme fee | NGN 45,000 |
| Payment status | Paid |

We look forward to welcoming you and wish you a rewarding learning experience. Congratulations on your admission.

**Dr. Mnena Ukeyima**
Deputy Director, Programmes & Training

consultancy@moaum.edu.ng · +234 803 000 0000 · Verify this letter online with reference MOAUM-PR-2026-5WQ3A

*Figure C.1: Generated letter of admission (PDF)*

# APPENDIX D: TEST CASES AND RESULTS

All tests were executed by the group on the local demonstration system and, where indicated, on the live deployment. The demonstration accounts used are listed in Appendix F.

**Table D.1: Test cases and results**

| ID | Area | Test performed | Expected result | Result |
|---|---|---|---|---|
| T01 | Public | Open all public pages on the live site | Each page loads (HTTP 200) with data from the database | Pass |
| T02 | Public | View shop availability by campus and category | Shops listed with live status; filters work | Pass |
| T03 | Public | Track application MOAUM-SA-2026-9RT4W | Status timeline and committee note shown | Pass |
| T04 | Accounts | Sign in as student, staff and admin | Each lands on the dashboard with the correct role | Pass |
| T05 | Accounts | Sign in as admin on the live site | Admin console opens | Pass |
| T06 | Accounts | Open /dashboard and /admin when signed out | Redirected to the sign-in page | Pass |
| T07 | Shops | Apply for shop EC-A01 with ID and proposal files | Application saved, reference issued, files stored privately | Pass |
| T08 | Shops | Choose an Eastern Campus shop from its card | Form pre-selects campus and shop | Pass |
| T09 | Shops | Apply twice for the same campus | Second application refused | Pass |
| T10 | Shops | Allocate the application as admin | Status Allocated; shop marked Occupied; statistics updated | Pass |
| T11 | Shops | Download allocation letter | Valid PDF with crest and shop details | Pass |
| T12 | Courses | Register for Graphic Design Essentials | Registration saved with a reference; payment instructions shown | Pass |
| T13 | Courses | Submit a bank transfer reference | Payment status becomes Awaiting Verification | Pass |
| T14 | Courses | Staff use Verify payment & confirm | Registration Confirmed and Paid; admission letter available | Pass |
| T15 | Payments | Complete a Paystack test-mode checkout of NGN 15,000 | Registration automatically Confirmed and Paid | Pass |

| ID | Area | Test performed | Expected result | Result |
|---|---|---|---|---|
| T16 | Payments | Simulated failed charge / underpayment / wrong currency | Payment refused in each case | Pass |
| T17 | Payments | Same successful payment reported twice | Second report has no effect | Pass |
| T18 | Payments | Webhook with missing, forged or tampered signature | Rejected (HTTP 401) | Pass |
| T19 | Notify | Submit a contact enquiry | Acknowledgement to sender and enquiry to the Unit inbox | Pass |
| T20 | Notify | Staff change an application's status | Applicant emailed with the new status and note | Pass |
| T21 | Notify | Send a test email through Gmail SMTP | Email received in the inbox | Pass |
| T22 | Reports | Export applications and registrations as CSV | CSV files downloaded with all records | Pass |
| T23 | Security | Student opens /admin and /admin/users | Redirected to the student dashboard | Pass |
| T24 | Security | Student requests another user's document and letter | Refused (HTTP 403) | Pass |
| T25 | Security | Student requests the CSV export | Refused (HTTP 403) | Pass |
| T26 | Security | Staff open the users page (admin only) | Redirected; no user data in the response | Pass |
| T27 | Security | Use a session belonging to a deleted account | Treated as signed out | Pass |
| T28 | Security | Submit a form before the page finishes loading | No personal data appears in the URL | Pass |
| T29 | Layout | Measure nine public pages at 375 px width | No horizontal scrolling | Pass |
| T30 | Build | Run the TypeScript check and production build | Both complete without errors | Pass |
| T31 | Deploy | Production seed run twice; short admin password | Catalogue and one admin created once; short password refused; no demo accounts | Pass |
| T32 | Deploy | Push to GitHub main branch | Vercel builds and publishes the new version | Pass |

# APPENDIX E: PROJECT STRUCTURE

The source code is organised as follows.

```
moaum-consultancy/
├── prisma/
│   ├── schema.prisma          database model
│   ├── catalog.ts             shops and programmes catalogue
│   ├── seed.ts                demonstration data (development)
│   └── seed-production.ts     catalogue + first administrator (production)
├── public/brand/              official crest images
├── scripts/
│   ├── prepare-postgres.mjs   switches to PostgreSQL on Vercel
│   └── test-email.ts          checks the SMTP settings
├── src/
│   ├── app/
│   │   ├── (site)/            public pages: home, services, shops, programmes,
│   │   │                      about, contact, login, register
│   │   ├── dashboard/         applicant dashboard
│   │   ├── admin/             admin console
│   │   └── api/               auth, files, letters, CSV export, Paystack
│   ├── components/            UI components (ui, layout, home, shops, ...)
│   ├── data/site.ts           site content: contacts, FAQs, guidelines
│   ├── lib/                   auth, database client, validation, constants
│   ├── server/                Server Actions, mail, payments, letters, uploads
│   └── middleware.ts          route protection
├── docs/diagrams/             UML and architecture diagrams
├── .env.example               configuration template
└── package.json               dependencies and scripts
```

# APPENDIX F: INSTALLATION AND DEPLOYMENT GUIDE

## F.1 Running the Portal Locally

Requirements: Node.js 20 or later and npm. In the project folder:

```
npm install
copy .env.example .env      (then set NEXTAUTH_SECRET)
npm run setup               (creates the database and demo data)
npm run dev                 (opens on http://localhost:3000)
```

## F.2 Demonstration Accounts (local only)

**Table F.1: Demonstration accounts**

| Email | Role | Password |
|---|---|---|
| admin@moaum.edu.ng | Administrator | Password123 |
| staff@moaum.edu.ng | Staff | Password123 |
| student@moaum.edu.ng | Student | Password123 |

These accounts exist only in the local demonstration database. They are never created on the live site, whose single administrator account is configured privately during deployment.

## F.3 Configuration Settings

**Table F.2: Environment variables**

| Variable | Purpose |
|---|---|
| DATABASE_URL | Database connection (SQLite file locally; PostgreSQL on Vercel) |
| NEXTAUTH_URL, NEXTAUTH_SECRET | Site address and the secret used to sign sessions |
| SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD | Outgoing email server and credentials (Gmail App Password) |
| MAIL_FROM, MAIL_UNIT_INBOX | Sender name/address and the inbox for enquiries |
| PAYSTACK_SECRET_KEY | Paystack secret key (test or live) |
| ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NAME | First administrator created on deployment |
| BLOB_READ_WRITE_TOKEN | Vercel Blob storage (added automatically by Vercel) |

### F.4 Deploying to Vercel

1. Import the GitHub repository into Vercel (Add New → Project).
2. Under Storage, create a Neon PostgreSQL database and a Blob store and connect both to the project.
3. Under Settings → Environment Variables, add the variables in Table F.2.
4. Deploy. Every later push to the main branch is built and published automatically.
5. In the Paystack dashboard, set the webhook URL to https://moaum-consultancy.vercel.app/api/payments/paystack/webhook.